

Combinational Circuits

Digital Logic Design

FY. BTech



Introduction

- Logic circuits for digital systems may be combinational or sequential.
- A logic circuit is combinational if its outputs at any time are a function of only the present inputs.
- A combinational circuit performs an operation that can be specified logically by a set of Boolean functions.
- In contrast, sequential circuits employ storage elements in addition to logic gates.
- In sequential circuits outputs are a function of the inputs and the state of the storage elements. Because the state of the storage elements is a function of previous inputs to the circuit, the outputs of a sequential circuit depend at any time on not only the present values of inputs but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states.

Combinational Circuit

- A combinational circuit consists of an interconnection of logic gates.
- Combinational logic gates react to the values of the signals at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data.
- A block diagram of a combinational circuit is shown in Figure below.
- The n input binary variables come from an external source; the m output variables are produced by the input signals acting on the internal combinational logic circuit, and go to an external destination.
- Each input and output variable exists physically as an analog signal¹ whose values are interpreted to be a binary signal that represents logic 0 and logic 1.

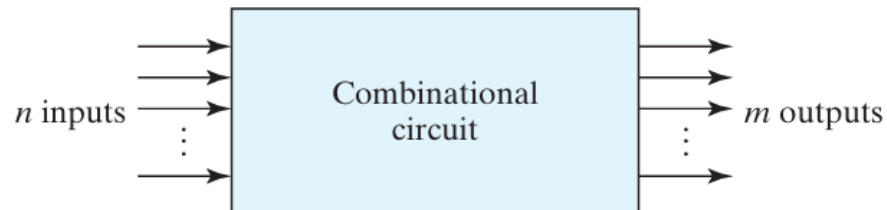


FIGURE 4.1

Block diagram of combinational circuit

COMPARISON

Aspect	Combinational Circuit	Sequential Circuit
Definition	Output depends only on the current inputs.	Output depends on both current inputs and past states (memory).
Memory Elements	Does not require memory elements.	Requires memory elements like flip-flops or latches.
Timing Dependency	Output is immediate, based on input changes.	Output is dependent on clock pulses and previous states.
Clock Signal	No clock signal required.	Requires a clock signal to synchronize state changes.
Design Complexity	Simpler design without the need for memory.	More complex due to memory and clock management.
Speed	Faster, as outputs change instantly with inputs.	Slower due to dependency on clock cycles and past states.
Functionality	Performs basic logical operations without sequence dependency.	Performs operations that require sequences or timed events.
Examples	Adders, Subtractors, Multiplexers, Encoders.	Counters, Shift Registers, Flip-Flops, State Machines.
Power Consumption	Generally lower power consumption.	Higher power consumption due to memory and clock circuitry.

Binary Adder-Subtractor

- Digital computers perform a variety of information-processing tasks. Among the functions encountered are the various arithmetic operations.
- The most basic arithmetic operation is the addition of two binary digits.
- A combinational circuit that performs the addition of two bits is called a half adder.
- One that performs the addition of three bits (two significant bits and a previous carry) is a full-adder.
- The names of the circuits stem from the fact that two half adders can be employed to implement a full adder.
- A binary adder–subtractor is a combinational circuit that performs the arithmetic operations of addition and subtraction with binary numbers.
- The subtraction circuit is included in a complementing circuit.

HALF ADDER

- A half adder circuit has two binary inputs and two binary outputs.
- A combinational circuit that performs the addition of two bits is called a half adder.
- The input is considered as 'x' and 'y' the circuit has two outputs sum 'S' and carry 'C'
- The truth table is shown below:

Table 4.3
Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

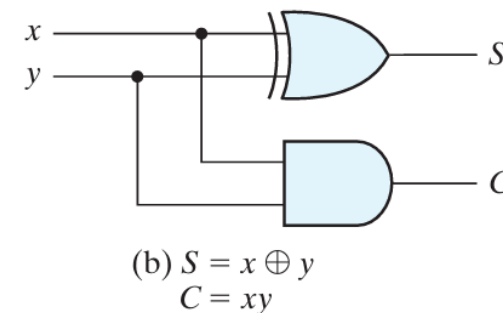
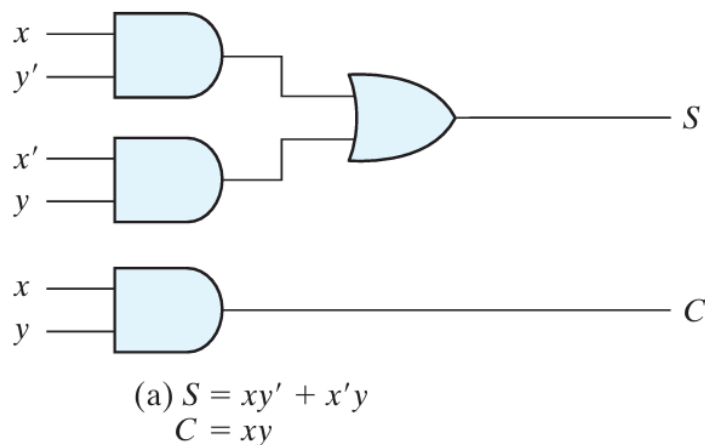


FIGURE 4.5
Implementation of half adder

HALF SUBTRACTOR

- A logic circuit for the subtraction of **B** (subtrahend) from **A** (minuend) where **A** and **B** are 1-bit numbers is referred to as a half-subtractor.
- The subtraction process is illustrated below and is reproduced in truth table form.
- Here, A and B are the two inputs and **D** (difference) and **C** (borrow) are the two outputs.

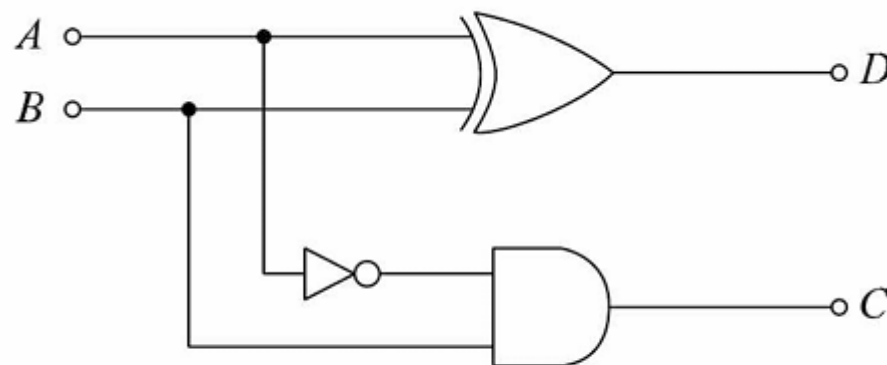


Fig. 5.22 *Realisation of a Half-subtractor*

HALF SUBTRACTOR

Table 5.13 *Truth Table of a Half-subtractor*

Inputs		Outputs	
<i>A</i>	<i>B</i>	<i>D</i>	<i>C</i>
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

From the truth table, the logical expressions for *D* and *C* are obtained as

$$D = \bar{A}B + A\bar{B} = A \oplus B \quad (5.36a)$$

$$C = \bar{A}B \quad (5.36b)$$

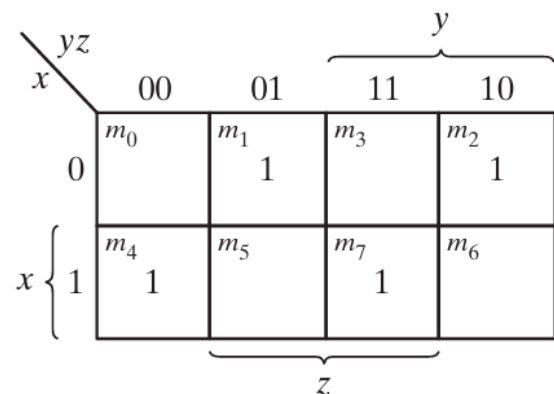
FULL ADDER

- A full adder is a combinational circuit that forms the arithmetic sum of three bits.
- It consists of three inputs and two outputs.
- Two of **the input variables**, denoted by 'x' and 'y', represent the two significant bits to be added. The third input, 'z', represents **the carry from the previous lower significant position**.
- Two outputs are necessary because the arithmetic sum of three binary digits ranges in decimal value from 0 to 3, and binary representation of the decimal digits 2 or 3 needs two bits.
- The two outputs are designated by the symbols '**S**' for sum and '**C**' for carry.
- The binary variable '**S**' gives the value of the **least significant bit of the sum**.
- The binary variable '**C**' gives the **output carry** formed by adding the input carry and the bits of the words.

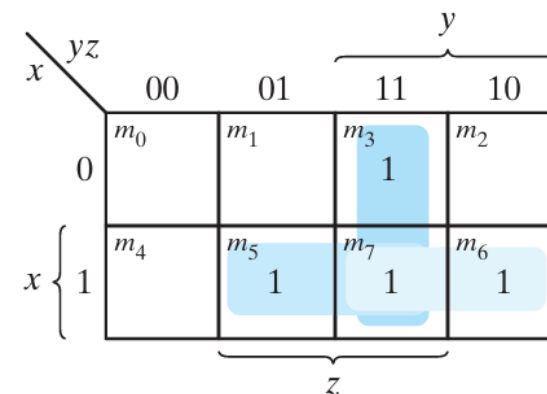
FULL ADDER (cont.)

Table 4.4
Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$(a) S = x'y'z + x'yz' + xy'z' + xyz$$



$$(b) C = xy + xz + yz$$

FIGURE 4.6
K-Maps for full adder

FULL ADDER (cont..)

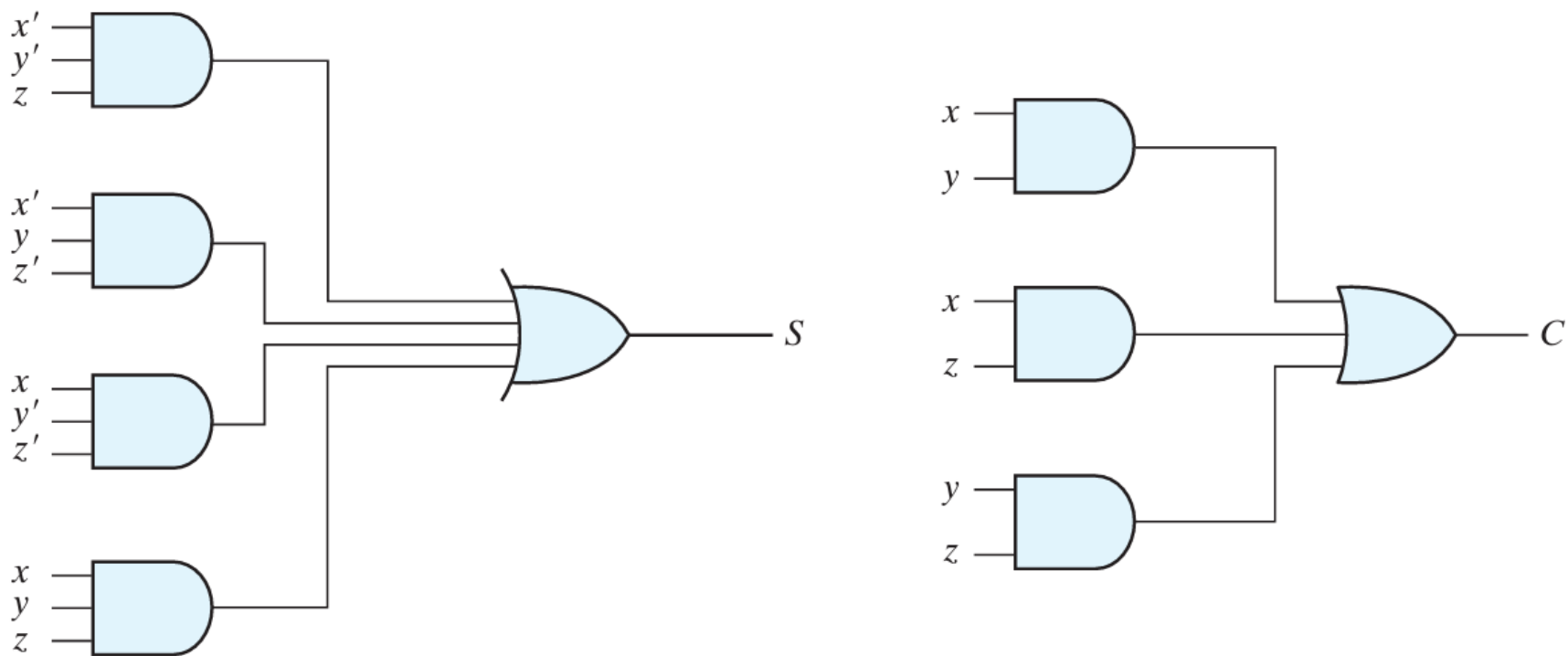


FIGURE 4.7

Implementation of full adder in sum-of-products form

FULL SUBTRACTOR

- Just like a full-adder, a full-subtractor circuit is required for performing multibit subtraction wherein a borrow from the previous bit position may also be present.
- A full-subtractor features three inputs: A_n (minuend), B_n (subtrahend), and C_{n-1} (borrow from the previous stage).
- The circuit produces two outputs: D_n (difference) and C_n (borrow).
- The K-map for the output D_n is identical to the K-map for S_n of the adder circuit.

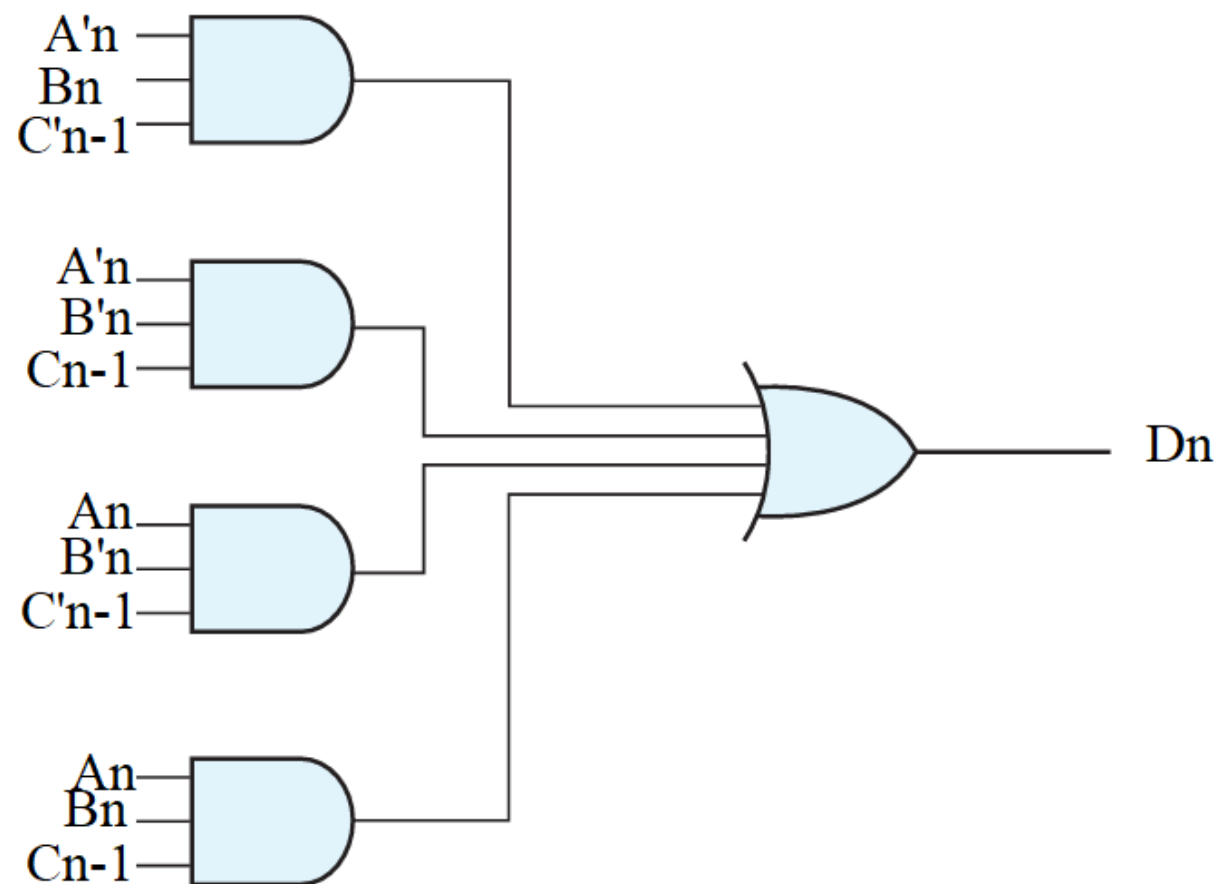
FULL SUBTRACTOR (cont.)

		$A_n B_n$			
		00	01	11	10
C_{n-1}	0		1		1
	1	1		1	

K-map for D_n

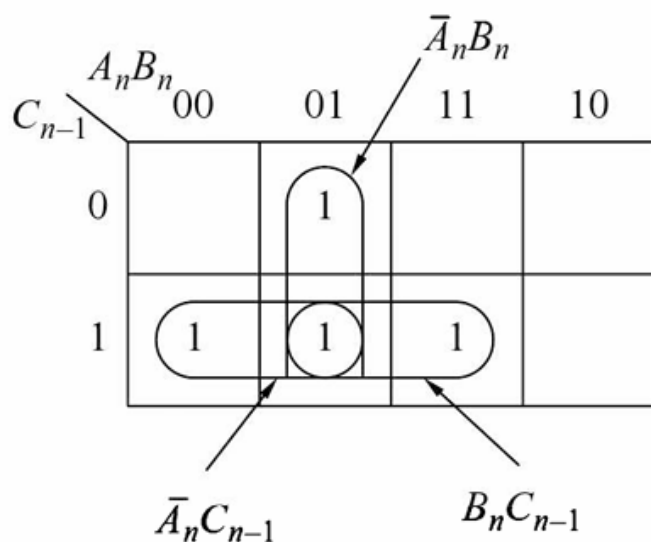
(a)

$$D_n = \bar{A}_n B_n \bar{C}_{n-1} + \bar{A}_n \bar{B}_n C_{n-1} + A_n \bar{B}_n \bar{C}_{n-1} + A_n B_n C_{n-1}$$

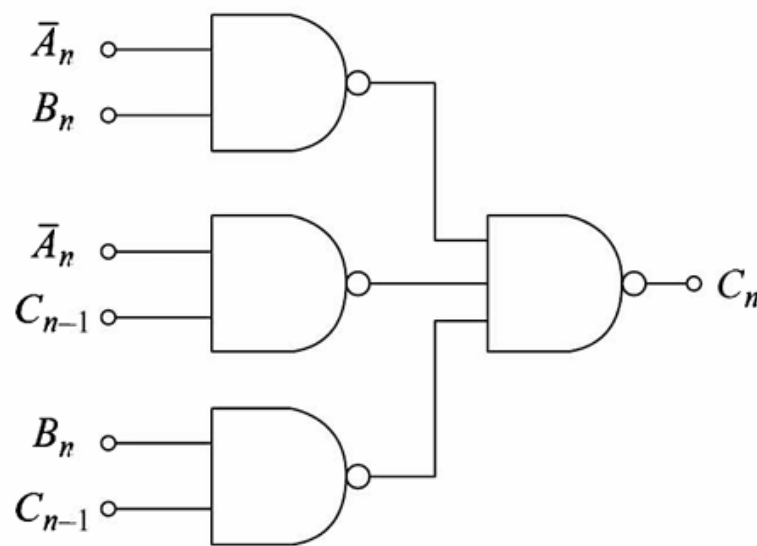


FULL SUBTRACTOR (cont..)

$$C_n = \bar{A}_n B_n + \bar{A}_n C_{n-1} + B_n C_{n-1}$$



(a)



(b)

Fig. 5.23

(a) K-map for C_n (b) Realisation of C_n

FULL ADDER USING TWO HALF ADDERS

Implement Full Adder using two half adder & a basic gates.

Solution:-

Truth table for Full Adder

A	B	Cin	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

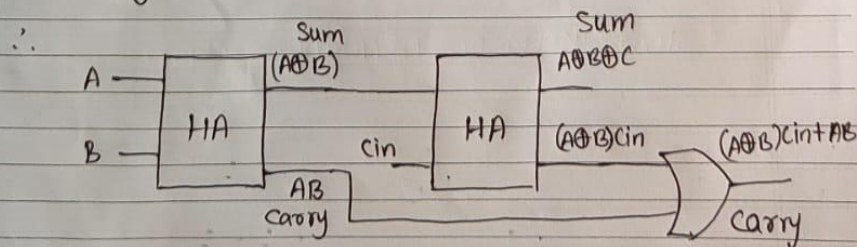
$$\begin{aligned} \text{Sum} &= \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} \\ &= C_{in}(\bar{A}\bar{B} + \bar{A}B + A\bar{B} + AB) \\ &= C_{in}(A \oplus B) + C_{in}(A \oplus B) \\ &= A \oplus B \oplus C \end{aligned}$$

$$\begin{aligned} \text{Carry} &= \bar{A}BC_{in} + \bar{A}B\bar{C}_{in} + AB\bar{C}_{in} + ABC_{in} \\ &= C_{in}(\bar{A}B + AB) + AB(\bar{C}_{in} + C_{in}) \\ &= C_{in}(A \oplus B) + AB \end{aligned}$$

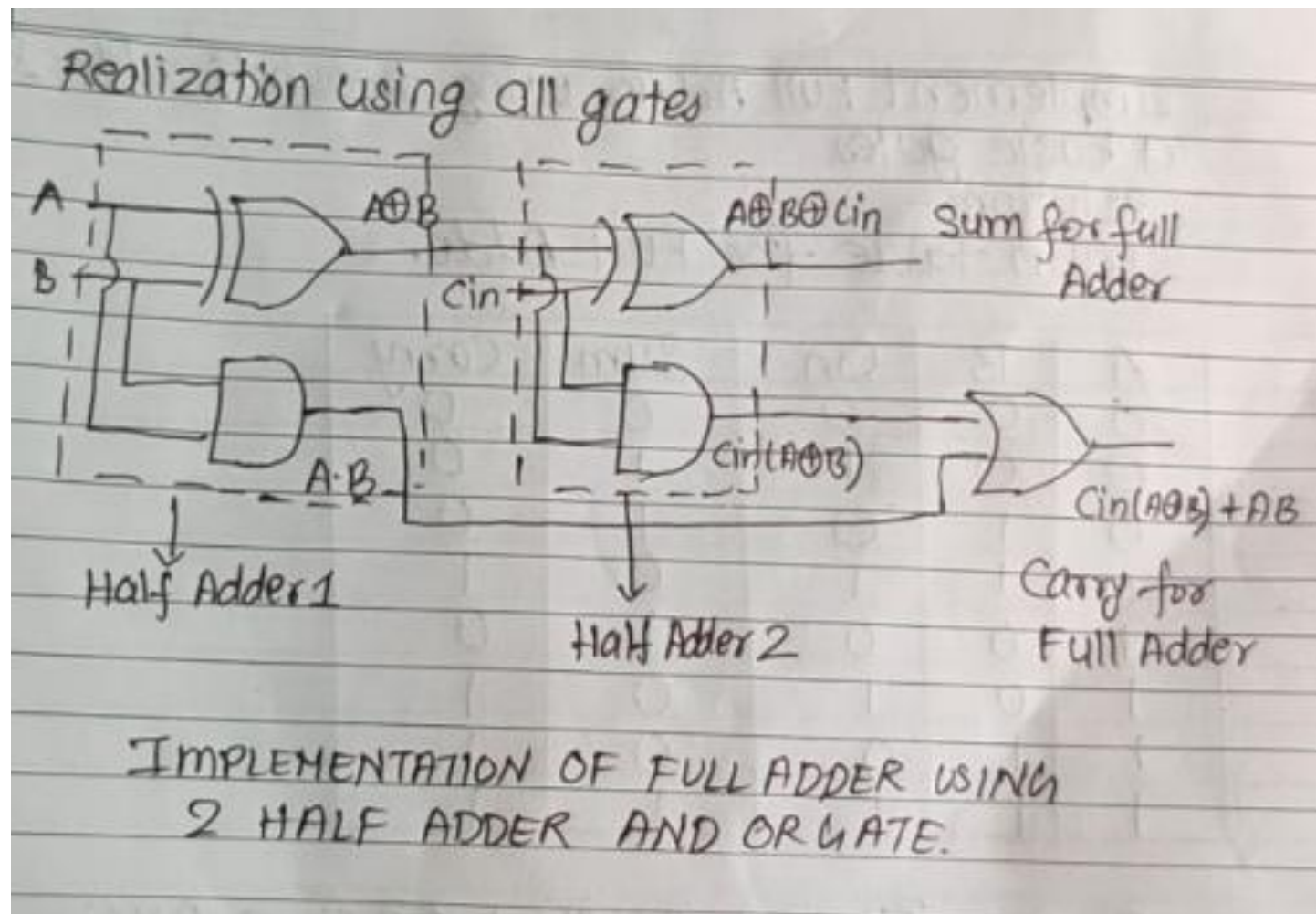
But we know that for half adder

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = AB$$



FULL ADDER USING TWO HALF ADDERS



FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

Inputs			Outputs	
A	B	Bin	Difference ^D	Bout Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

Truth Table for Difference (D):

A		B Bin			
		00	01	11	10
0	0	0	1	0	1
1	0	1	0	1	0

Difference $D = \bar{A}\bar{B}Bin + \bar{A}B\bar{Bin} + A\bar{B}\bar{Bin} + ABBin$

Truth Table for Borrow (Bout):

A		B Bin			
		00	01	11	10
0	0	0	1	1	1
1	0	0	1	1	0

Borrow $Bout = \bar{A}Bin + \bar{A}B + BBin$

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

The borrow output for circuit shown, can be given as

$$B_{out} = \bar{A} B_{in} + \bar{A} B + B B_{in}$$

$$= \bar{A} B_{in} (B + \bar{B}) + \bar{A} B + B B_{in} (A + \bar{A})$$

$$= \bar{A} B B_{in} + \bar{A} \bar{B} B_{in} + \bar{A} B + A B B_{in} + \bar{A} B B_{in}$$

$$= \bar{A} B (B_{in} + 1 + B_{in}) + \bar{A} \bar{B} B_{in} + A B B_{in}$$

$$= \bar{A} B + \bar{A} \bar{B} B_{in} + A B B_{in}$$

$$= \bar{A} B + B_{in} (\bar{A} \bar{B} + A B)$$

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR

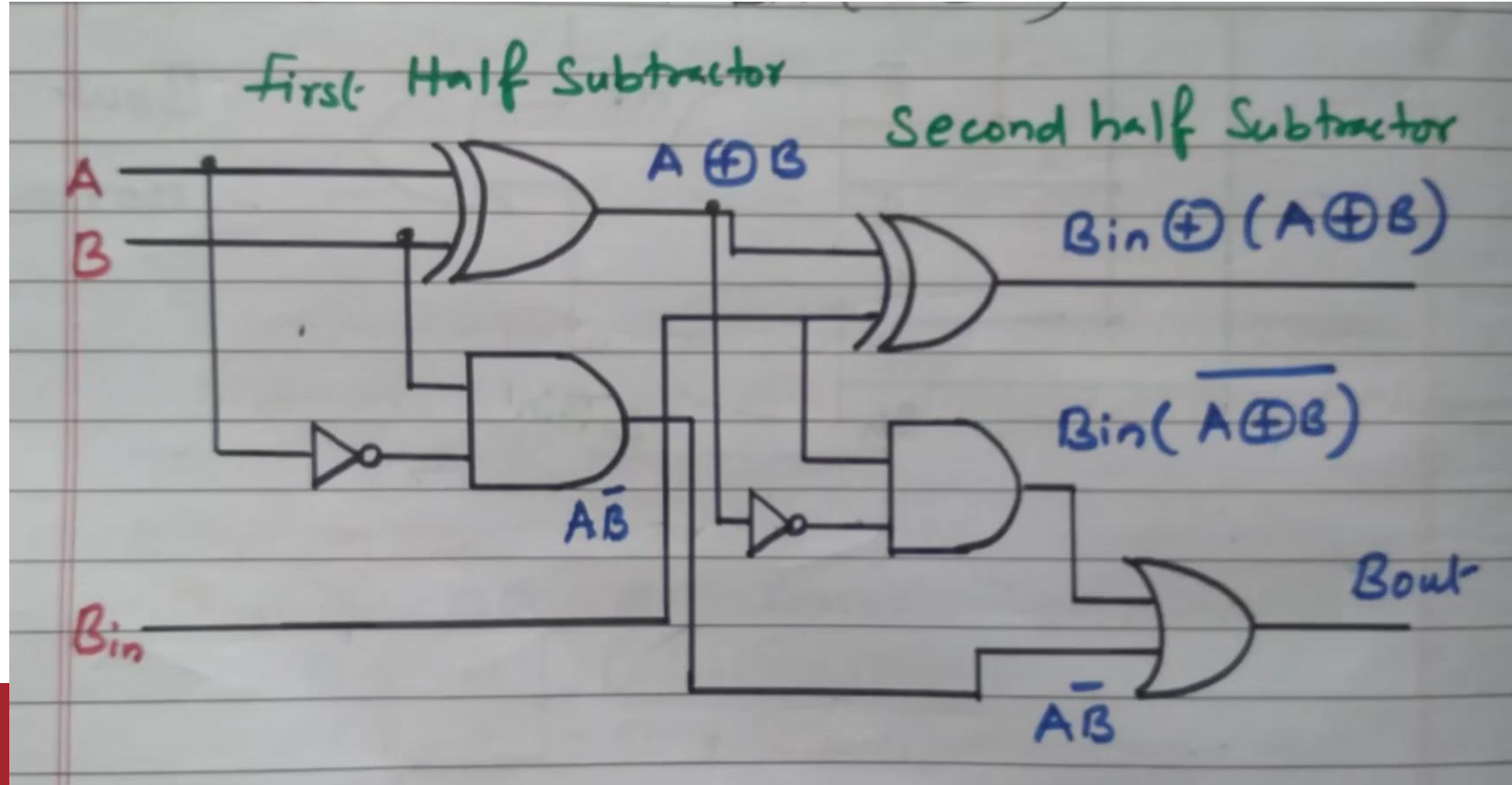
$$\text{Bout} = \overline{A}B + B\text{in}(A \oplus B)$$

A full Subtractor can also be implemented with two half Subtractors and one OR gate

$$D = B\text{in} \oplus (A \oplus B)$$

$$\text{Bout} = \overline{A}B + B\text{in}(\overline{A \oplus B})$$

FULL SUBTRACTOR USING TWO HALF SUBTRACTOR



CONTROLLED ADDER/SUBTRACTOR

- A Controlled Adder Subtractor is essentially the "Swiss Army Knife" of binary arithmetic.
- Instead of having one circuit for addition and a totally separate one for subtraction, a clever trick is used to combine both into a single piece of hardware.
- The circuit uses a Control Signal (usually labeled as M or Mode) to decide which operation to perform

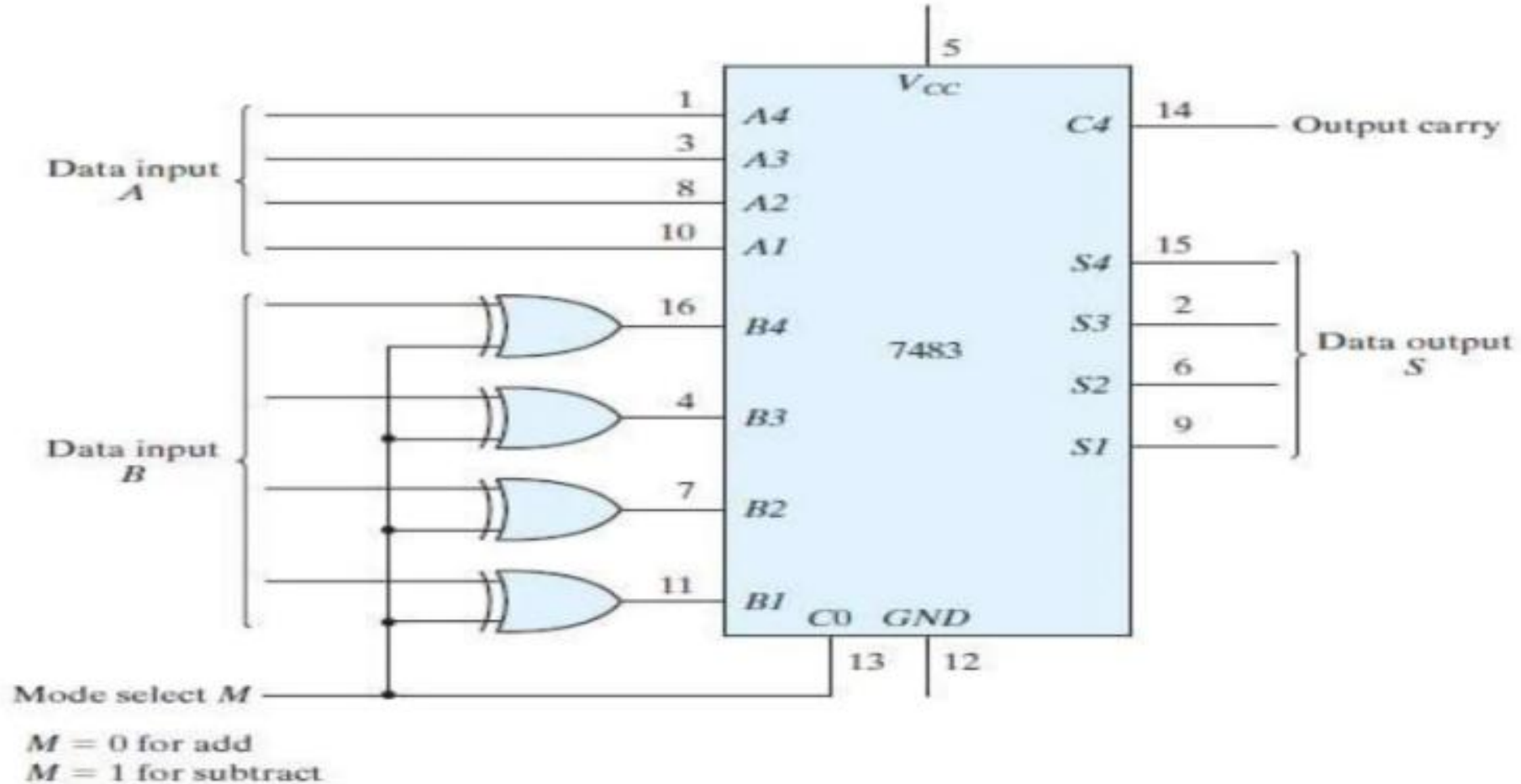
How It Works: The "Magic" of the XOR Gate

- ❑ The heart of this circuit is the XOR gate.
- ❑ In this setup, the XOR gate acts as a conditional inverter.

Remember the XOR rules for an input B and a control signal M:

1. If $M = 0$: $B \oplus 0 = B$ (The input stays the same).
2. If $M = 1$: $B \oplus 1 = \bar{B}$ (The input is inverted/flipped).

IC 7483



CONTROLLED ADDER/SUBTRACTOR (cont.)

1. Addition Mode ($M = 0$)

- When you set the control signal to 0:
- **The XOR Gates:** They pass the B inputs through without changing them ($B \oplus 0 = B$)
- **The Carry-In (C_{in}) :** The M signal is also connected to the very first carry-in of the adder chain.
- So, $C_{in} = 0$.
- **The Result:** The circuit performs a standard binary addition:
- **Sum = A + B + 0**

CONTROLLED ADDER/SUBTRACTOR (cont.)

1. Addition Mode ($M = 0$)

- When you set the control signal to 0:
- **The XOR Gates:** They pass the B inputs through without changing them ($B \oplus 0 = B$)
- **The Carry-In (C_{in}) :** The M signal is also connected to the very first carry-in of the adder chain.
- So, $C_{in} = 0$.
- **The Result:** The circuit performs a standard binary addition:
- **Sum = A + B + 0**

CONTROLLED ADDER/SUBTRACTOR (cont..)

2. Subtraction Mode (M = 1):

When you flip the control signal to 1, the circuit uses 2's Complement logic to subtract:

The XOR Gates: They flip all the bits of B ($B \oplus 1 = \bar{B}$).

This gives us the 1's complement.

The Carry-In (C_{in}): Since $M = 1$, the first carry-in is now 1.

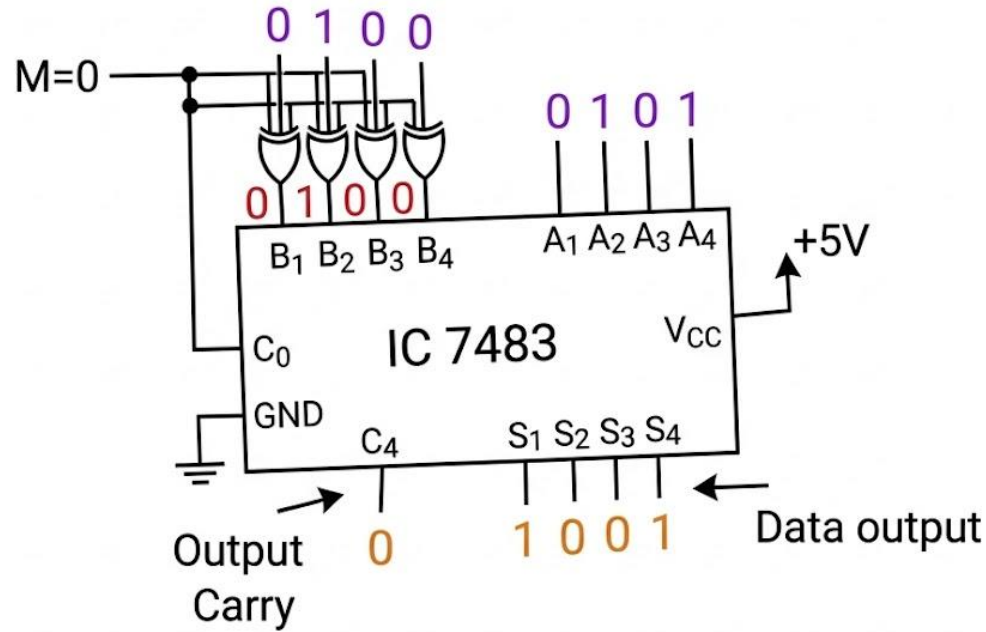
The Result: Adding 1 to the 1's complement gives us the **2's complement**.

The circuit performs : **Sum = $A + \bar{B} + 1$**

(Which mathematically is exactly the same as $A - B$)

IC 7483

Example for Addition



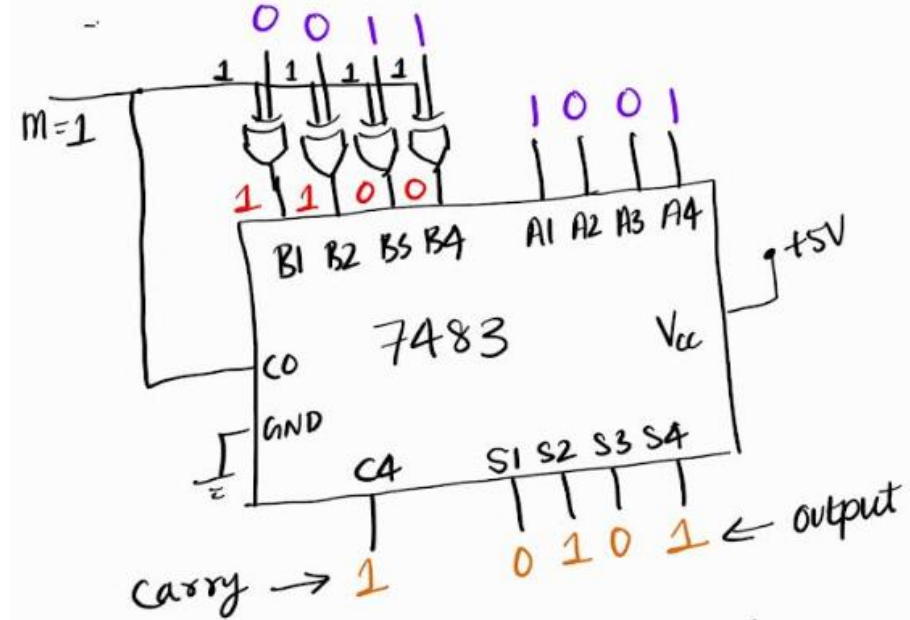
Eg:

$$\begin{array}{r} 5 \\ + 4 \\ \hline = 9 \end{array}$$

$$\begin{array}{r} 0101 \\ + 0100 \\ \hline (1001)_2 \end{array}$$

$$\begin{array}{r} 0100 \\ + 0101 \\ \hline (1001)_2 \end{array}$$

IC 7483 example for subtraction



Let us consider subtraction of

$$\begin{array}{r} 9 \\ - 3 \\ \hline 6 \end{array}$$

$$\begin{array}{r} 1001 \\ 0011 \\ \hline (0110)_2 \end{array}$$

$$\begin{array}{r} 1001 \\ 1100 \\ \hline 0101 \\ \boxed{1} \\ \hline 0110 \end{array}$$

CARRY
WILL BE
ADDED
DUE TO
ONE'S
COMPLEMENT

BCD ADDER

- BCD stands for binary coded decimal.
- It is used to perform the addition of BCD numbers.
- BCD digit can have any of ten possible four-bit representations. Suppose, we have two 4-bit numbers A and B.
- The value of A and B can vary from 0(0000 in binary) to 9(1001 in binary) because we are considering decimal numbers.
- The output will vary from 0 to 18 if we are not considering the carry from the previous sum.
- But if we are considering the carry, then the maximum value of output will be 19 (i.e. $9+9+1 = 19$).
- When we are simply adding A and B, then we get the binary sum. Here, to get the output in BCD form, we will use BCD Adder.

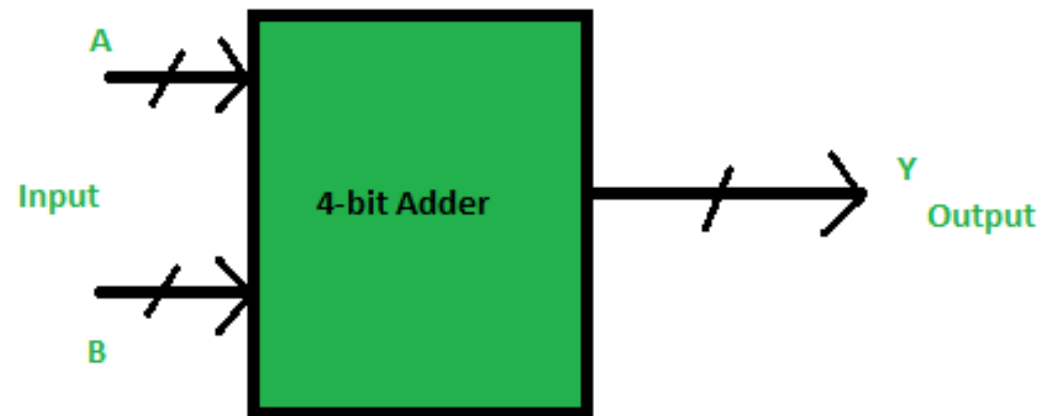


Figure : Block Diagram of BCD Adder

BCD ADDER

Decimal	Binary Sum					BCD Sum				
	C'	S3'	S2'	S1'	S0'	C	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

Figure : Table of BCD Adder

BCD ADDER

We are adding "0110" (=6) only to the second half of the table. **The conditions are:**

- 1.If $C' = 1$ (Satisfies 16-19)
- 2.If $S3'.S2' = 1$ (Satisfies 12-15)
- 3.If $S3'.S1' = 1$ (Satisfies 10 and 11)

So, our logic is

$$C' + S3'.S2' + S3'.S1' = 1$$

BCD ADDER

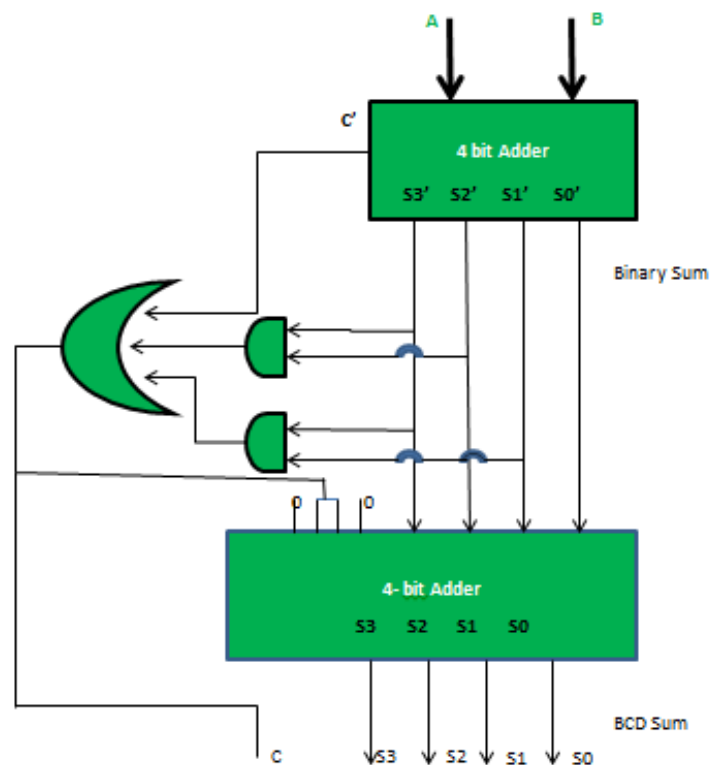


Figure: Implementation of BCD Adder.

RIPPLE CARRY ADDER

- A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers.
- It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain.
- Addition of n-bit numbers requires a chain of n full adders or a chain of one half adder and n- 1 full adders.
- In case of n full adders and one half adder the input carry to the least significant position is fixed at 0.
- The augend bits of A and the addend bits of B are designated by subscript numbers from right to left, with subscript 0 denoting the least significant bit.

RIPPLE CARRY ADDER

- The augend bits of A and the addend bits of B are designated by subscript numbers from right to left, with subscript 0 denoting the least significant bit.

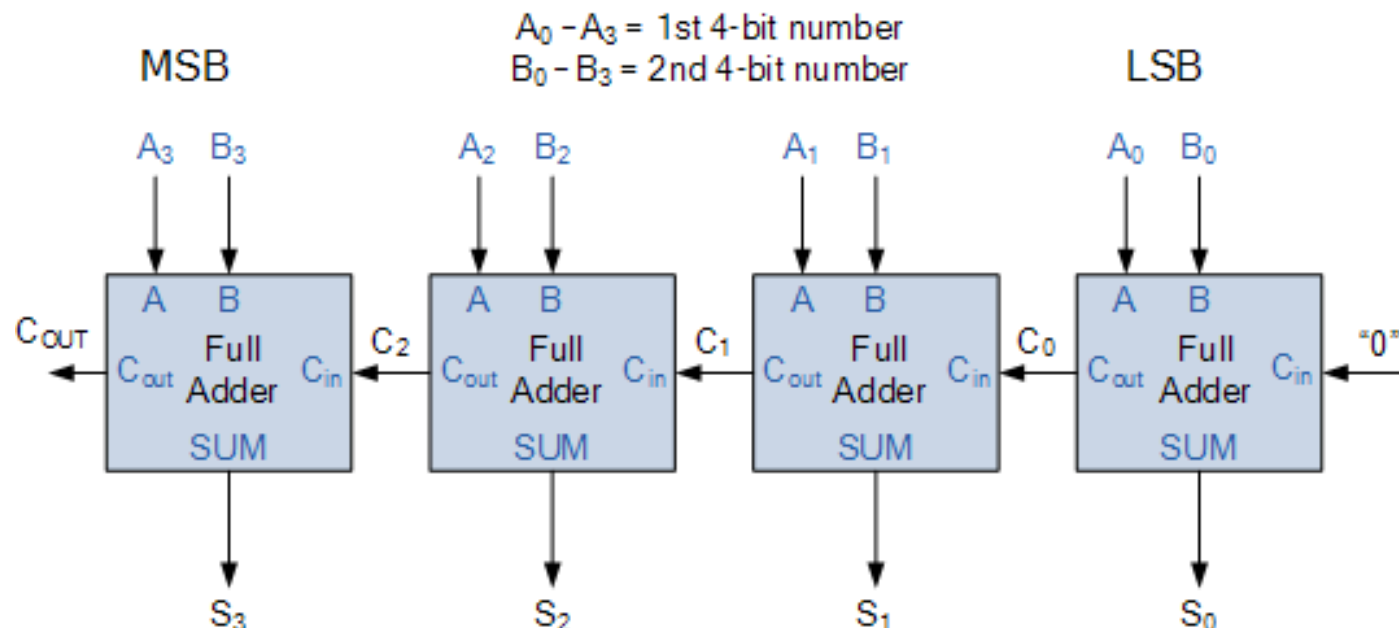


Figure: 4-bit adder

RIPPLE CARRY ADDER

- The carries are connected in a chain through the full adders.
- The input carry to the adder is C_0 , and it ripples through the full adders to the output carry C_4 . The S outputs generate the required sum bits.
- An n-bit adder requires n full adders, with each output carry connected to the input carry of the next higher order full adder.
- The sum bits are thus generated starting from the rightmost position and are available as soon as the corresponding previous carry bit is generated.
- All the carries must be generated for the correct sum bits to appear at the outputs.
- Note that the design of this circuit by the classical method would require a truth table with $2^9 = 512$ entries, since there are nine inputs to the circuit.

RIPPLE CARRY ADDER

- To demonstrate with a specific example, consider the two binary numbers $A = 1011$ and $B = 0011$. Their sum $S = 1110$ is formed with the four-bit adder as follows:

Subscript i:	3	2	1	0	
Input Carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Added	1	0	1	1	B_i
Sum	0	1	1	0	S_i
Carry	1	0	1	1	C_{i+1}

RIPPLE CARRY ADDER DIAGRAM

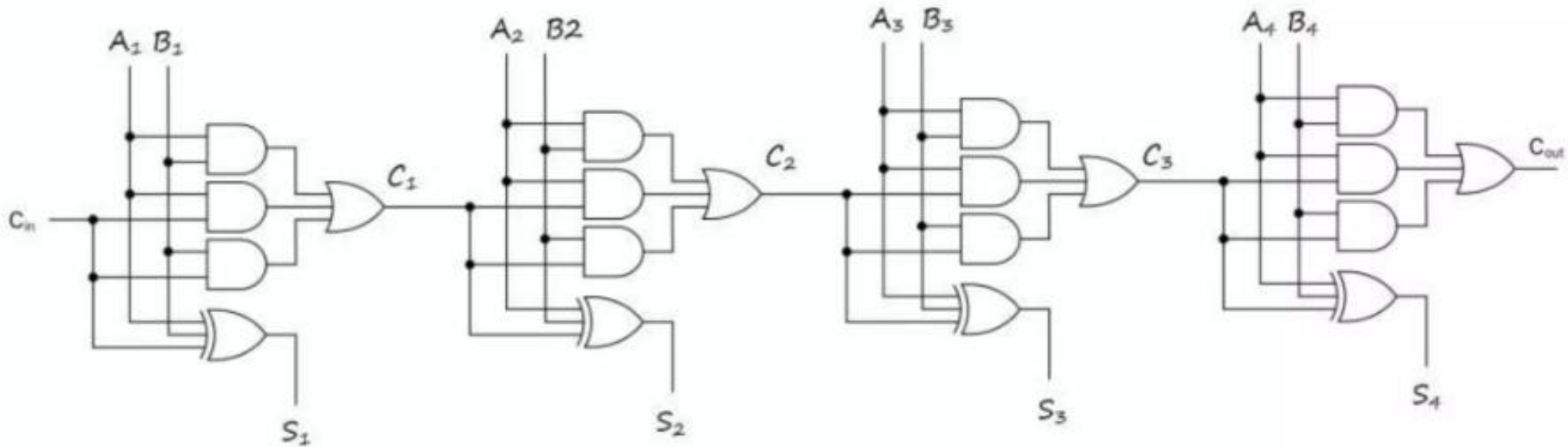


Figure: Ripple Carry Adder Realization using Logic gates.

PYQs

- State the difference between combinational and sequential circuits. -2023
- Implement Full Adder using Two Half Adders and few gates. -2023
- Design a 4-bit BCD Adder -2025
- Design and Implement 1 bit binary Full Adder using basic gates. -2025
- Design a Full Adder using half adder. -2021,2024
- Implement a full subtractor using two half subtractors. -2024,2023
- Implement a one digit BCD adder with a suitable diagram and equation. -2024
- Implement a Full adder with suitable diagram and truth table. -2024
- Explain logic for implementing BCD adder using 4 Bit adder Device (Binary adder). -2024
- Draw circuit diagram of controlled adder/subtractor using 4 bit adder IC and explain with example. -2024
- Differentiate between combinational and sequential circuit. -2024,2023,2022
- Design half adder using basic gates. -2024
- Design half adder realize using logic gates. -2024
- Design a full adder. Explain the working and application for the same. -2024

PYQs

- 2023
- 1. Implement a controlled adder/subtractor using 4 bit adder IC and additional gates (5 marks)
- 2. Implement a 4 bit adder using full adders (5 marks)
- 3. Draw the block diagram of a 4 bit adder/subtractor. Briefly explain how subtraction is done. (5 marks)
- 4. Design a 4 bit Ripple carry adder using full or half adder blocks. Label the circuit and explain the working using example $(1001)_2 + (1001)_2$ (10 marks)
- 5. Implement 4 bit adder/subtractor using MSI. (Pin numbers of MSI device are not expected). Explain operation with suitable example. (10 marks)

MULTIPLEXER

- A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line.
- The selection of a particular input line is controlled by a set of selection lines.
- Normally, there are 2^n input lines and n selection lines whose bit combinations determine which input is selected.
- A two-to-one-line multiplexer connects one of two 1-bit sources to a common destination, as shown in Figure on the next slide.

MULTIPLEXER

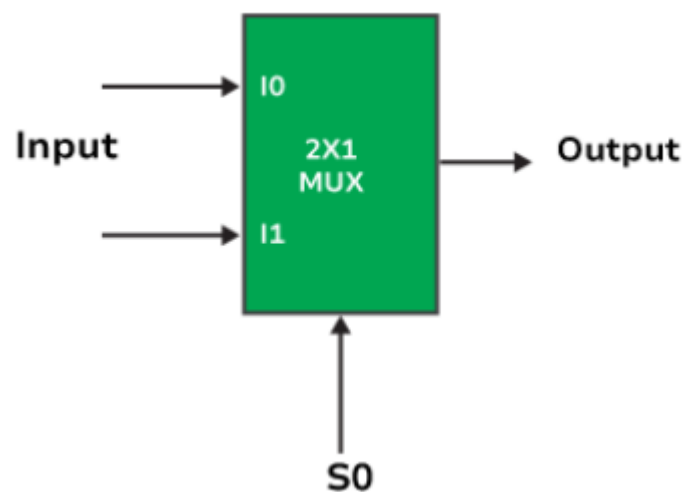


Figure a: Block Diagram of 2:1 multiplexer

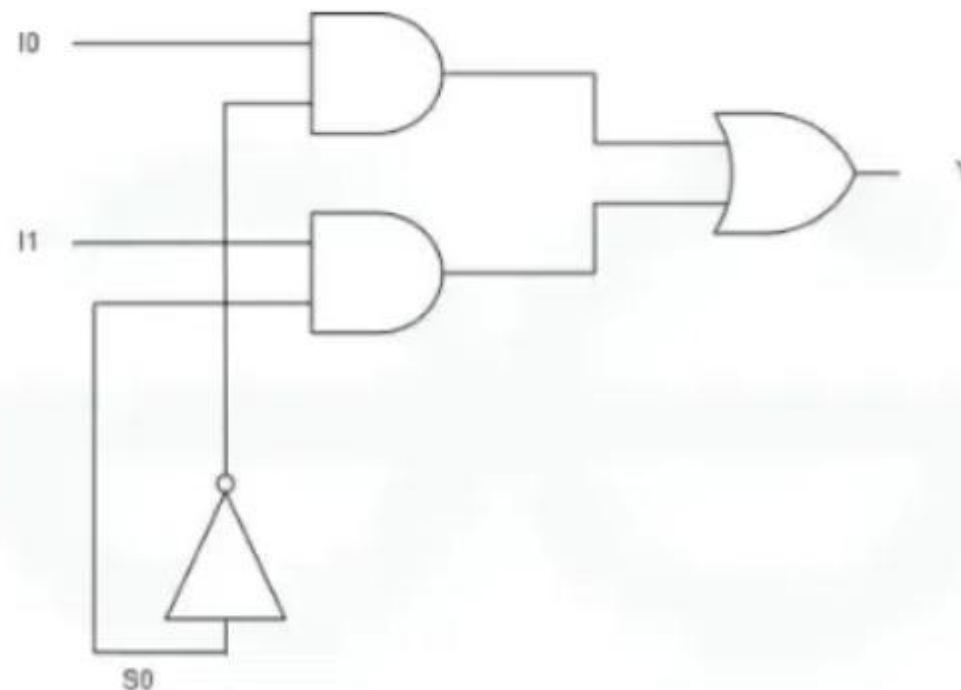


Figure b: Logic Gate realization of 2:1 multiplexer

WORKING OF 2:1 MUX

- The circuit has two data input lines, one output line, and one selection line S .
- When $S = 0$, the upper AND gate is enabled and I_0 has a path to the output.
- When $S = 1$, the lower AND gate is enabled and I_1 has a path to the output.
- The multiplexer acts like an electronic switch that selects one of two sources.
- The block diagram of a multiplexer is sometimes depicted by a wedge-shaped symbol, as shown in Fig. 4.24(b).
- It suggests visually how a selected one of multiple data sources is directed into a single destination.
- The multiplexer is often labeled “MUX” in block diagrams.

4:1 MULTIPLEXER

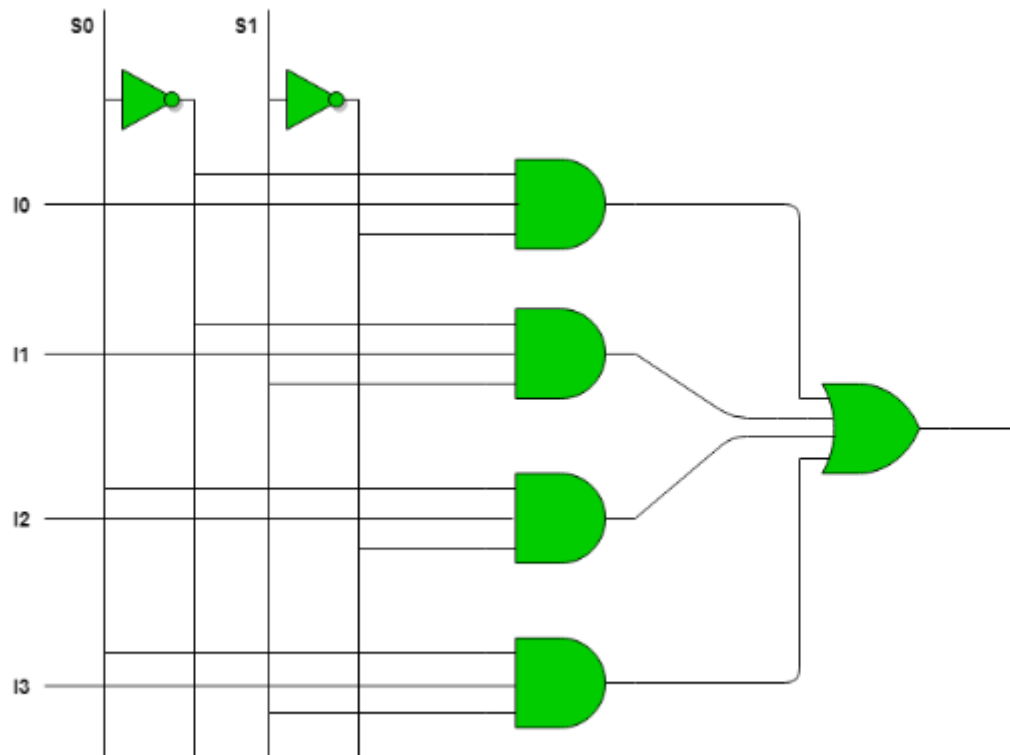


Figure a: Logic Gate realization of 4:1 multiplexer

Truth Table

S0	S1	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Figure b: Truth table of 4:1 multiplexer

DEMULTIPLEXER

A demultiplexer has

- ✓ N control inputs
- ✓ 1 data input
- ✓ 2^N outputs
- A demultiplexer routes (or connects) the data input to the selected output.
- The value of the control inputs determines the output that is selected.
- A demultiplexer performs the opposite function of a multiplexer.

DEMULTIPLEXER

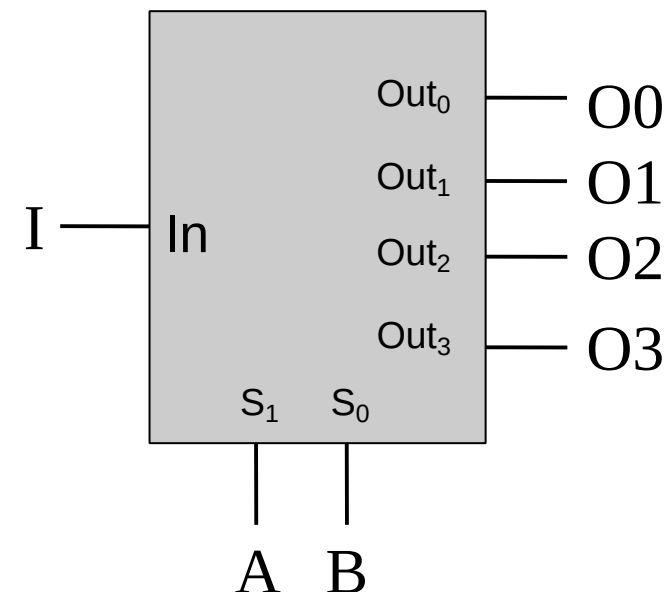
A	B	W	X	Y	Z
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

$$O0 = A'.B'.I$$

$$O1 = A.B'.I$$

$$O2 = A'.B.I$$

$$O3 = A.B.I$$



DEMULTIPLEXER

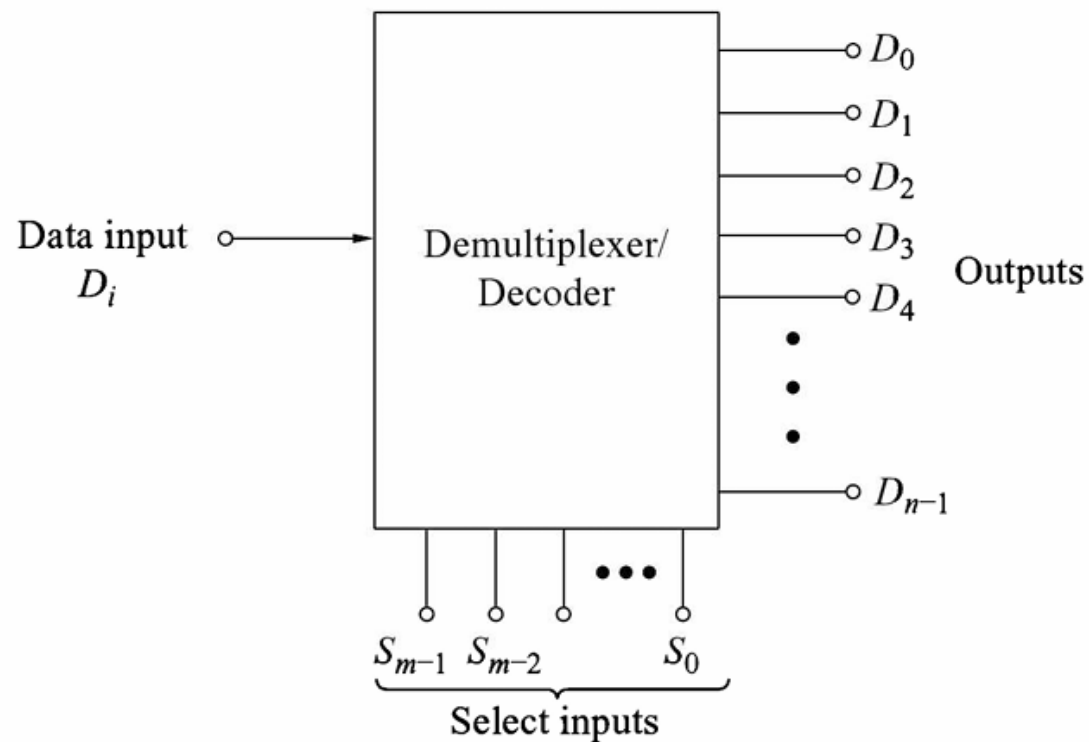


Fig. 6.7 *Block Diagram of a Demultiplexer.*

ENCODERS

- An encoder has
 - 2^N inputs
 - N outputs
- An encoder outputs the binary value of the selected (or active) input.
- An encoder performs the inverse operation of a decoder.
- Issues
 - What if more than one input is active?
 - What if no inputs are active?

ENCODERS

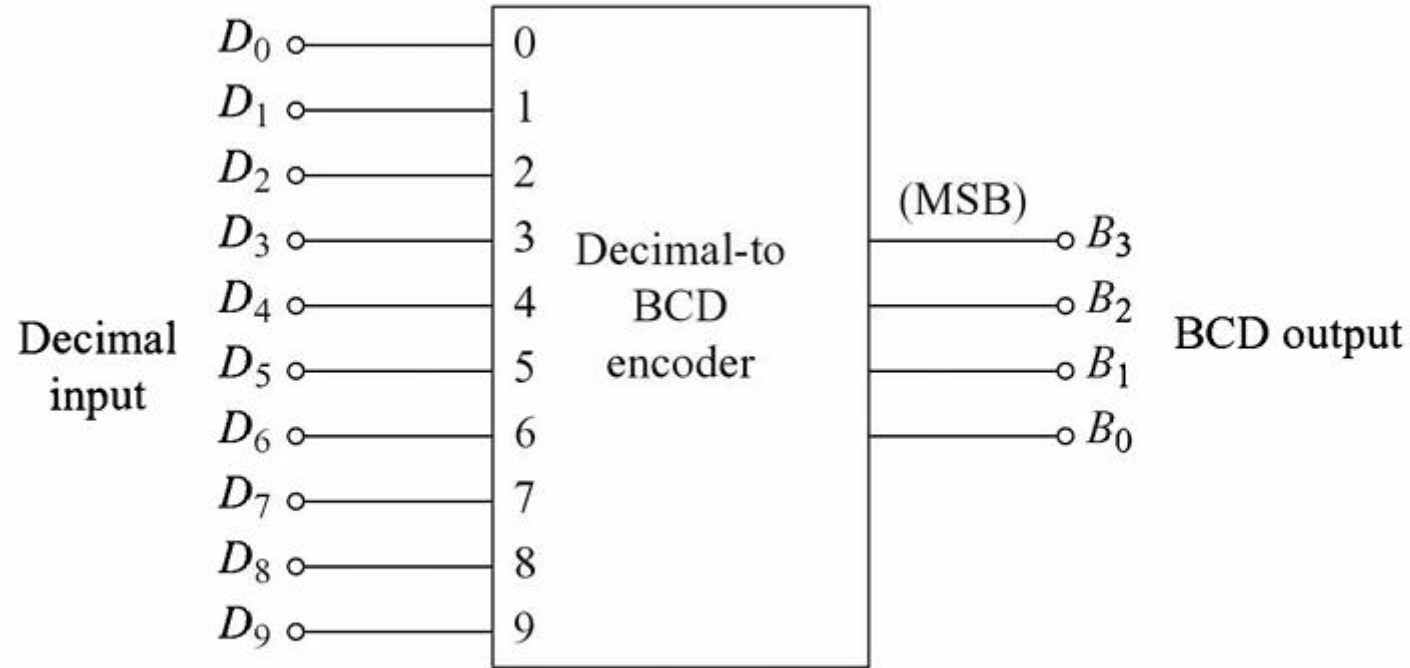


Fig. 5.27 *Block Diagram of Decimal-to-BCD Encoder*

ENCODERS

1. What if more than one input is active?

In a basic encoder, if two or more inputs are high (1) simultaneously, the output will be an **incorrect or "garbage" binary code** (usually a logical OR of the bits).

The Solution: Use a **Priority Encoder**.

How it works: This device assigns a priority level to each input line. If multiple inputs are active at the same time, the encoder only generates the binary code for the input with the **highest priority**. For example, if both input 3 and input 5 are active, a high-priority-wins encoder will output the binary code for 5 (101).

ENCODERS

2. What if no inputs are active?

If all inputs are 0, a standard encoder typically outputs all 0s (e.g., 00 for a 4-to-2 encoder). This creates an ambiguity: you **can't tell the difference between "Input 0 is active" and "No inputs are active"** because they both look like 00 at the output.

The Solution: A **Valid Bit (V)** output.

How it works: Designers add an extra output pin, often labeled **V (Valid)** or **GS (Group Signal)**.

V = 1: At least one input is active (the output is valid).

V = 0: No inputs are active (the output should be ignored).

PRIORITY ENCODER

- A priority encoder is an encoder circuit that includes the priority function.
- The operation of the priority encoder is such that if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence.
- In addition to the two outputs x and y , the circuit has a third output designated by V ; this is a valid bit indicator that is set to 1 when one or more inputs are equal to 1.
- **If all inputs are 0, there is no valid input and V is equal to 0.**
- **The other two outputs are not inspected when V equals 0 and are specified as don't-care conditions.**
- ❖ Note that whereas X 's in output columns represent don't-care conditions, the X 's in the input columns are useful for representing a truth table in condensed form.
- ❖ Instead of listing all 16 minterms of four variables, the truth table uses an X to represent either 1 or 0.

PRIORITY ENCODER

- According to Table 4.8, the higher the subscript number, the higher the priority of the input.
- Input D_3 has the highest priority, so, regardless of the values of the other inputs, when this input is 1, the output for xy is 11 (binary 3).
- D_2 has the next priority level. The output is 10 if $D_2=1$, provided that $D_3=0$, regardless of the values of the other two lower priority inputs.
- The output for D_1 is generated only if higher priority inputs are 0, and so on down the priority levels.

Table 4.8
Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

PRIORITY ENCODER

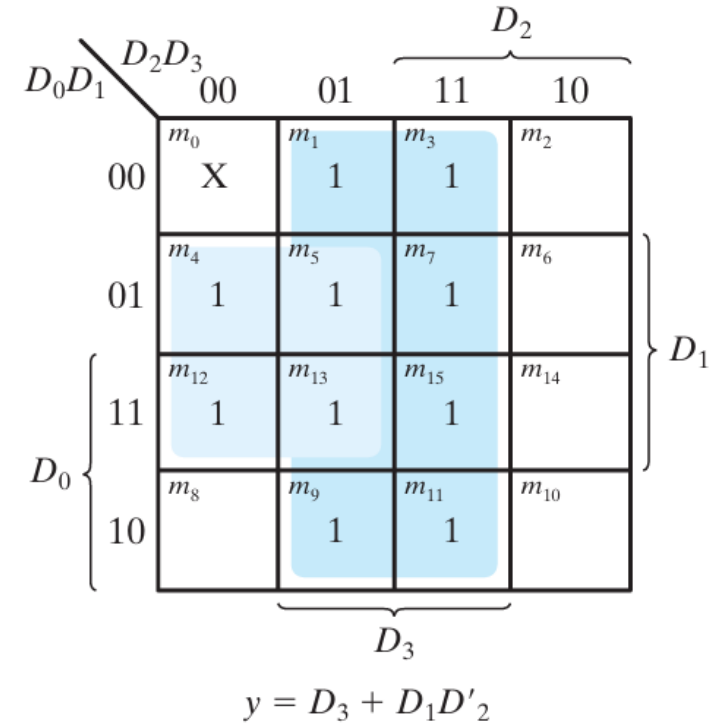
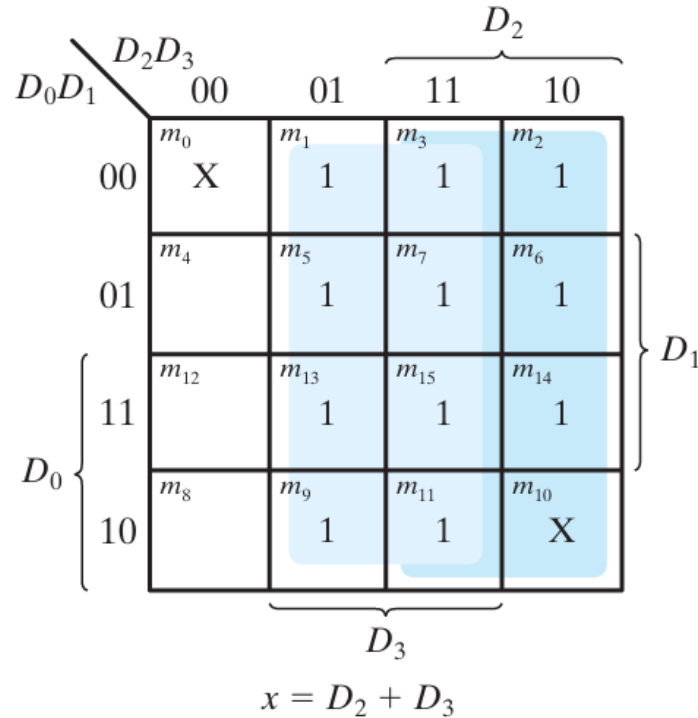


FIGURE 4.22

Maps for a priority encoder

PRIORITY ENCODER

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

$$V = D_0 + D_1 + D_2 + D_3$$

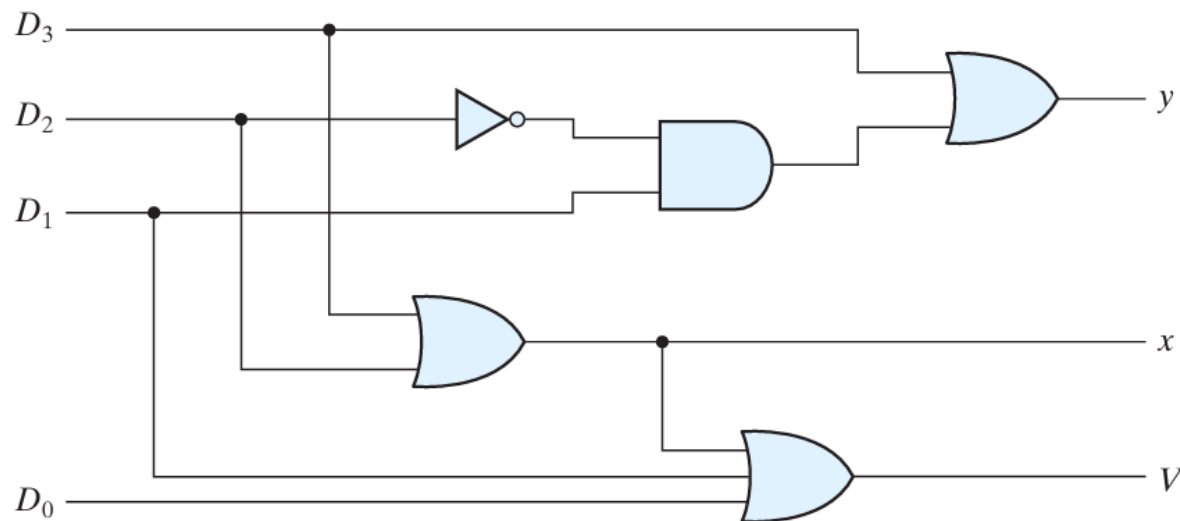


FIGURE 4.23
Four-input priority encoder

DECODER

- Discrete quantities of information are represented in digital systems by binary codes.
- A binary code of n bits is capable of representing up to 2^N distinct elements of coded information.
- A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^N unique output lines.
- If the n -bit coded information has unused combinations, the decoder may have fewer than 2^N outputs.
- The decoders presented here are called n -to- m -line decoders, where $m < 2^N$.
- Their purpose is to generate the 2^N (or fewer) minterms of n input variables.
- Each combination of inputs will assert a unique output.
- The name decoder is also used in conjunction with other code converters, such as a BCD-to-seven-segment decoder.
- A particular application of this decoder is binary-to-octal conversion. The input variables represent a binary number, and the outputs represent the eight digits of a number in the octal number system.

DECODER

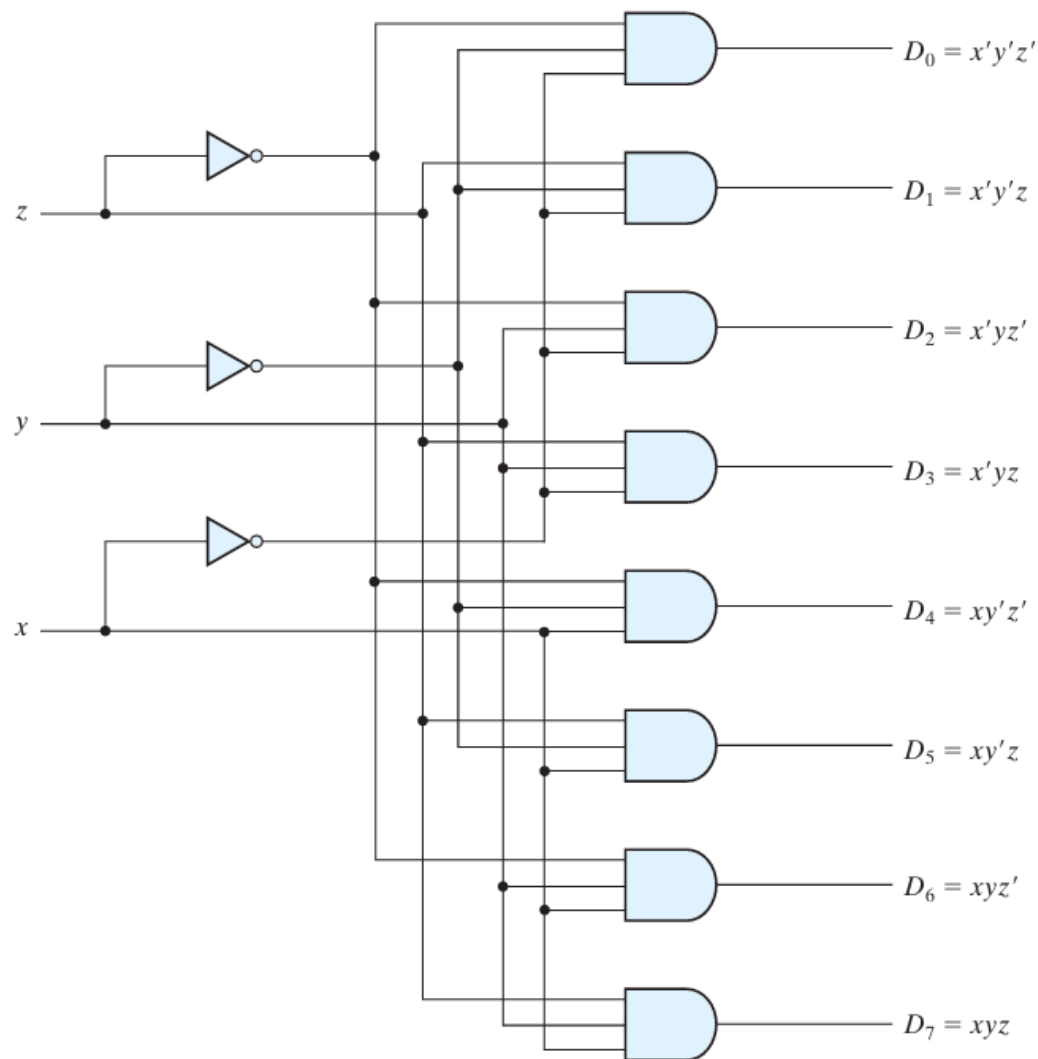


FIGURE 4.18
Three-to-eight-line decoder

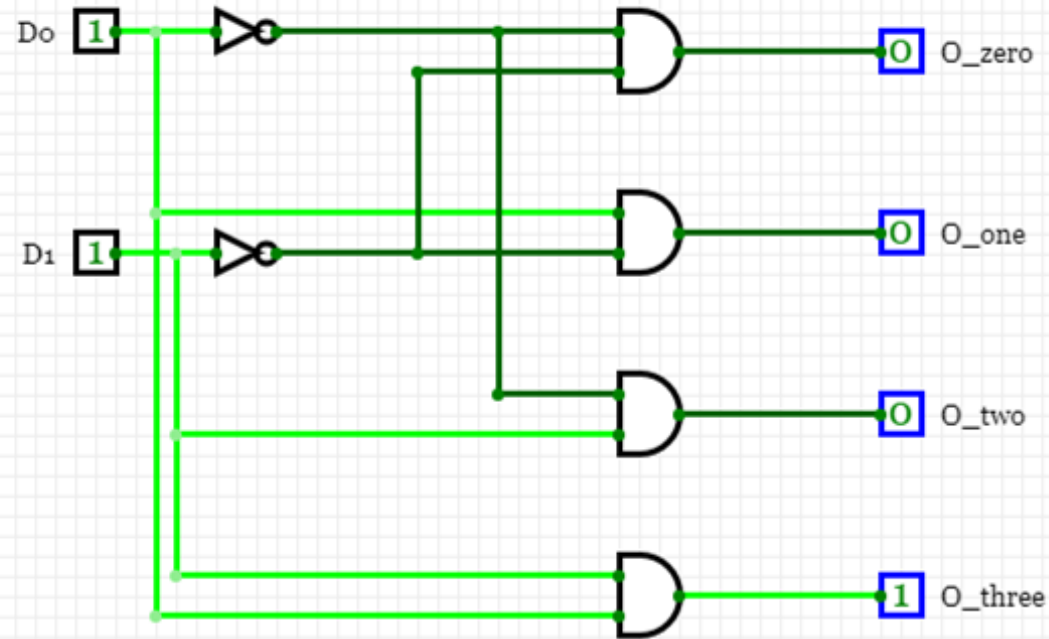
DECODER

Table 4.6

Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

2:4 Decoder



Decoder with enable input

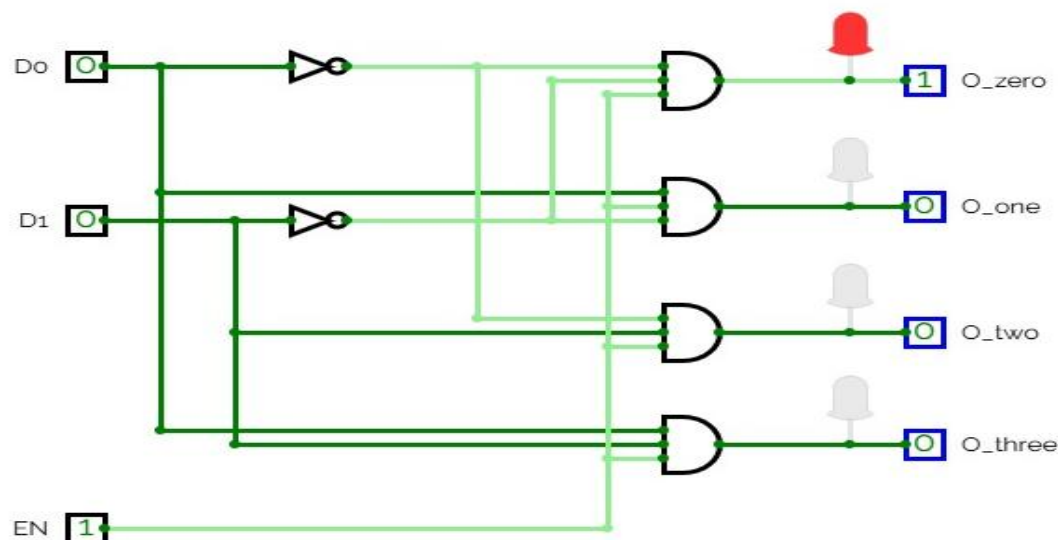
Minterm Generator

$$O_0 = E_n \cdot D_1' \cdot D_0'$$

$$O_1 = E_n \cdot D_1' \cdot D_0$$

$$O_2 = E_n \cdot D_1 \cdot D_0'$$

$$O_3 = E_n \cdot D_1 \cdot D_0$$



En	D1	D0	O3	O2	O1	O0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Decoder with active low output

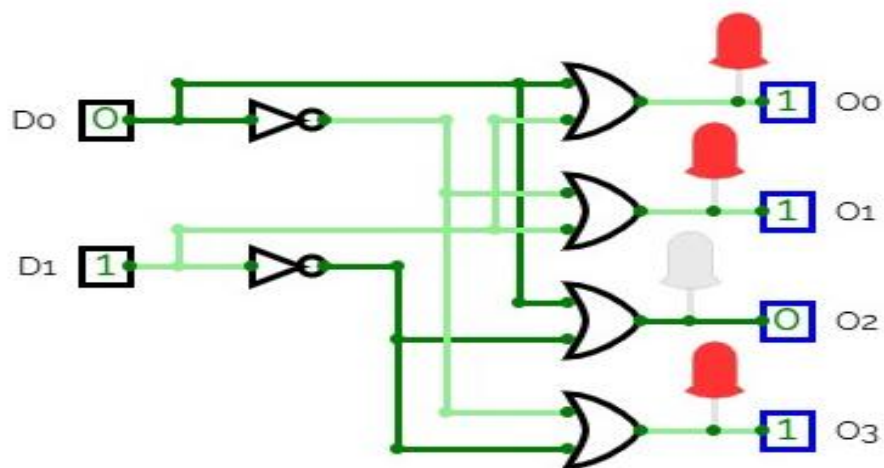
Maxterm Generator

$$O0 = (D1 + D0)$$

$$O1 = (D1 + D0')$$

$$O2 = (D1' + D0)$$

$$O3 = (D1' + D0')$$

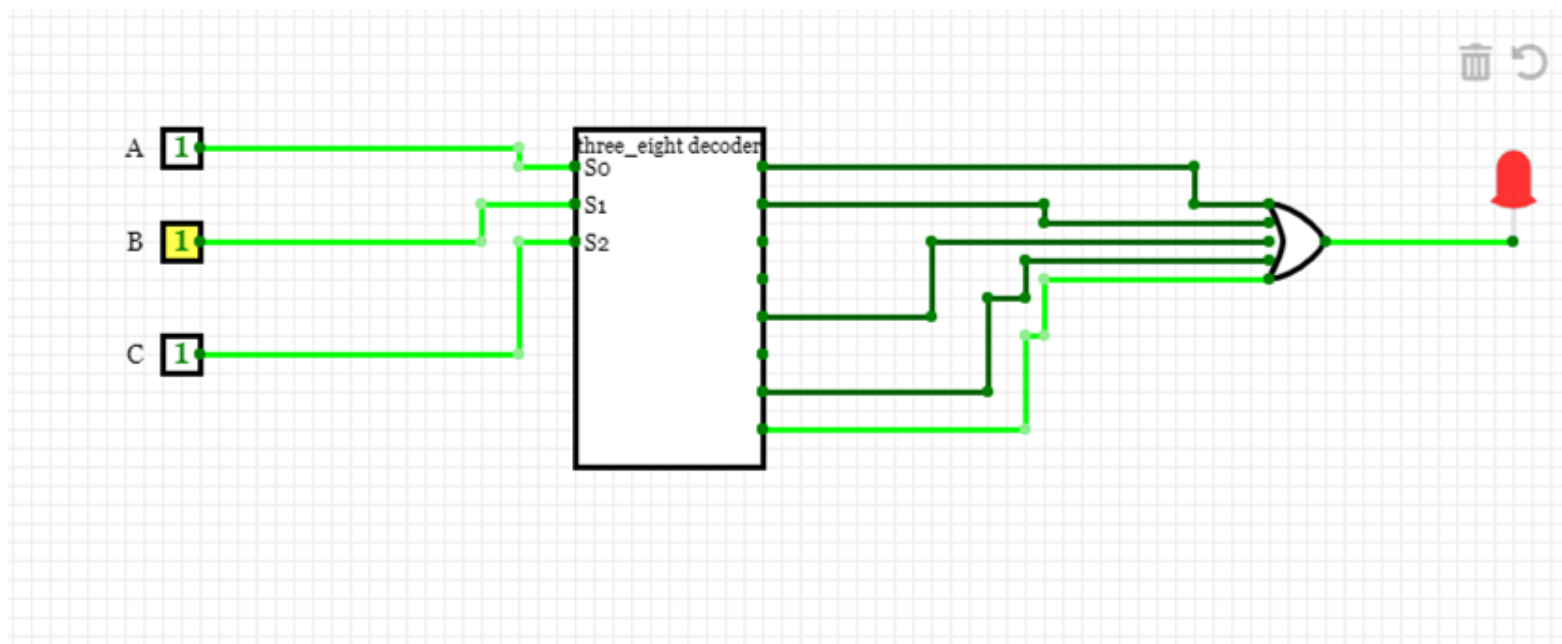


D1	D0	O3	O2	O1	O0
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

Function Implementation using Decoder

$$F = \Sigma m(0, 1, 4, 6, 7)$$

Implement using 3: 8 decoder(Active High Output)



NUMERICAL

- Implement 4:1 MUX using 2:1 MUX. -2021
- Implement the following expression using 4:1 Multiplexer $F(A,B,C) = \sum m(0,1,2,3,5,7)$ -2021
- Implement the following expression using MUX $F(A,B,C) = \sum m(0,1,3,6,7)$ -2025
- Implement the following expression using 4:1 MUX $f(A,B,C) = A'B'C' + A'B'C + A'BC + AB'C$ -(2020)
- Implement the following function $F(A,B,C,D) = \sum m(0,1,5,8,10,13,15)$ using:
1) 16:1 Multiplexer (2) only one 8:1 multiplexer -2025
- Design 4:1 multiplexer using basic gates. -2025
- Implement the following expression using 4:1 Multiplexer $F(A,B,C,D) = \sum m(0,2,3,5,11,12,15)$ -2024
- Implement the following expression using MUX $F = \prod M(0,1,3,6,7,9)$ -2024
- Realize using multiplexer $F(A,B,C) = \sum m(0,1,3,4,6)$ -2024
- $F = \sum m(0,2,4,5)$.Realise using 8:1 MUX -2024
- Design a Full Adder using a Multiplexer. -2024
- Implement $F(A,B,C,D) = \sum m(1,2,5,7,12,15)$ -2023
- Implement 8:1 MUX using 2:1 MUX -2023

NUMERICAL

- Implement $\Sigma m(0,1,3,6)$ 4:1 MUX tree -2023
- Implement the following function $F(A,B,C,D) = \Sigma m(2,4,5,7,10,14)$ using:
1) 16:1 Multiplexer (2) only one 8:1 multiplexer -2023
- Implement the following expression using single 8:1 Multiplexer $F(P,Q,R,S) = \Sigma m(0,1,3,5,7,8,11,13,14)$ -2023
- Implement $F(A,B,C,D) = \Sigma m(1,2,4,5,7,12,14)$ using a Multiplexer. -2023
- What is Multiplexer tree? Construct 32:1 multiplexer using 8:1 multiplexers only. Explain how the logic on particular data line is steered to the output in this design with example. -2023
- Write the Boolean expression for the output of a 2:1 multiplexer in terms of inputs and select lines. Draw the circuit using only NOR gates. -2023
- Draw logic diagram of 2:1 multiplexer and 1:2 Demultiplexer using Logic gates, write the truth tables as well as output expressions for both. -2023

MULTIPLEXER TREE

- Since 16-to-1 multiplexers are the largest available ICs, therefore, to meet the larger input needs there should be a provision for expansion.
- This is achieved with the help of enable/strobe inputs and multiplexers stacks or trees are designed.
- Two commonly used methods for this purpose are illustrated in Figures presented on the next slide

MULTIPLEXER TREE

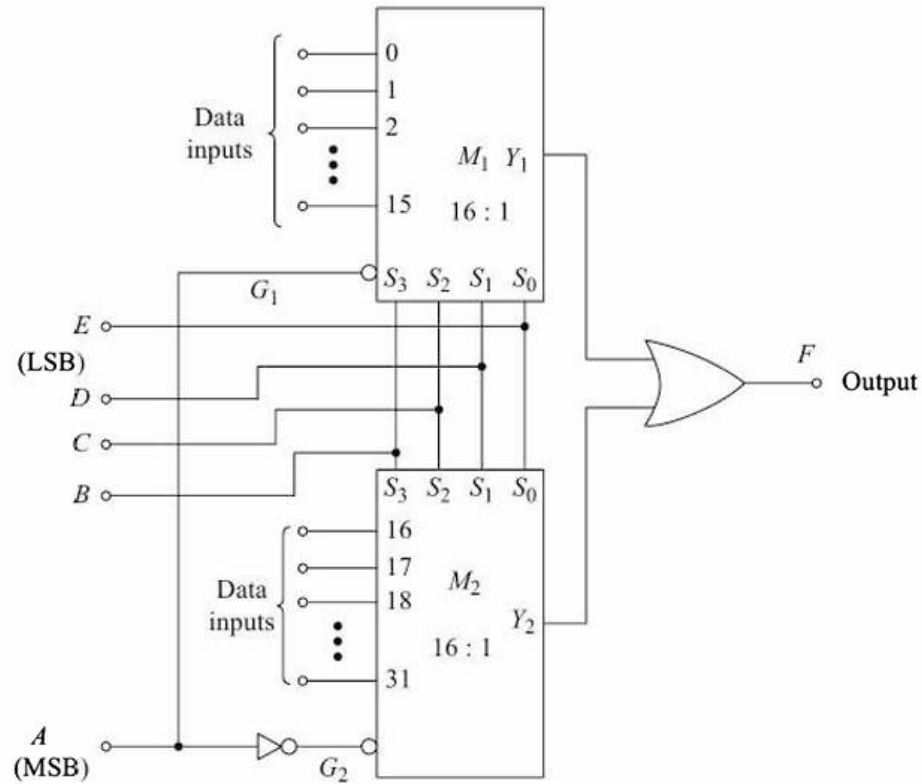


Fig. 6.5 32:1 Multiplexer Using Two 16:1 Multiplexers and One OR Gate

MULTIPLEXER TREE

- The circuit of Fig. 6.5 uses two 16:1 multiplexers (M_1 and M_2) for the realization of a 32:1 multiplexer.
- The lower order 16 data input lines ($I_0 - I_{15}$) are applied at the data input terminals of the multiplexer M_1 and the higher order 16 data input lines ($I_{16} - I_{31}$) are applied at the data input terminals of the multiplexer M_2 .
- For a 32:1 multiplexer, the number of select input lines is required to be 5.
- The 5-bit select inputs are 'ABCDE'. The most-significant select input bit 'A' is applied at G_1 and $\bar{A}$ is applied at G_2 .
- B, C, D , and E are connected to S_1, S_2, S_3 and S_0 inputs of both the multiplexers.
- The output F of the multiplexer is obtained by using an OR gate, where $F = Y_1 + Y_2$

MULTIPLEXER TREE

- The circuit of Fig. 6.6 uses a 2:1 multiplexer instead of an OR gate.
- When $A = 0$, $F = Y_1$ and when $A = 1$, $F = Y_2$.
- Thus, both the circuits perform similar operation.

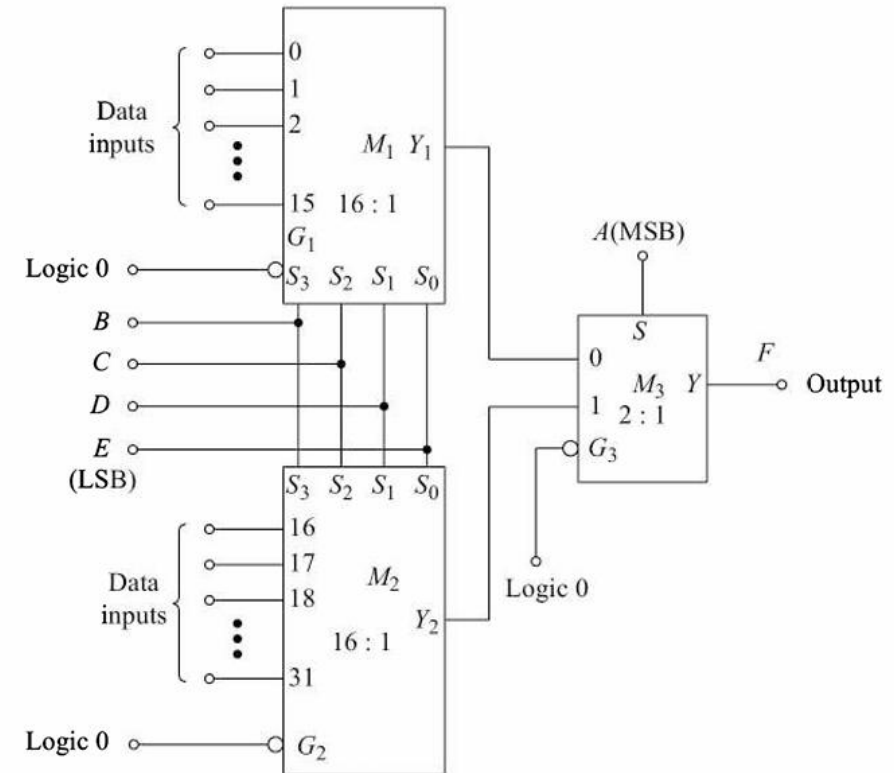


Fig. 6.6 32:1 Multiplexer Using Two 16:1 Multiplexers and One 2:1 Multiplexer

DECODER TREE

- Since 4-line-to-16-line decoders are the largest available circuits in ICs, to meet the larger inputs need there should be a provision for expansion.
- This is made possible by using enable input terminal.
- Figure 6.9 shows a 5-line-to-32-line decoder and Fig. 6.10 shows a 8-line-to-256-line decoder using 4-line-to-16-line decoders.
- In a similar way, any m-line-to-n-line decoder can be implemented.

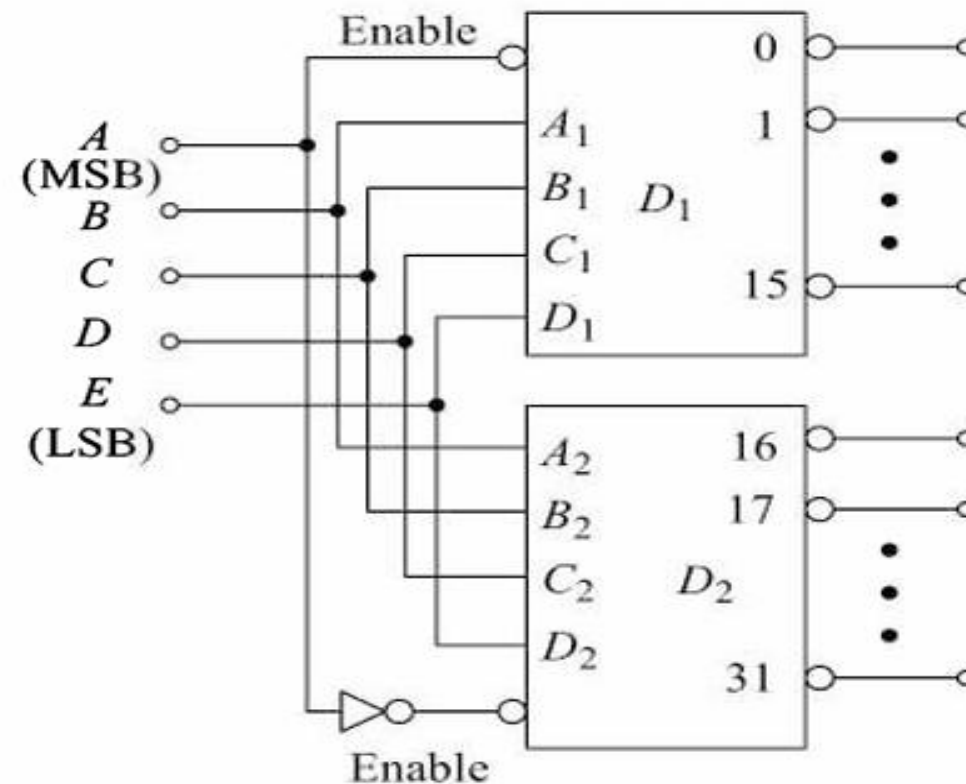


Fig. 6.9 **5-Line-to-32-Line Decoder Using Two 4-line-to-16-Line Decoders**

DECODER TREE

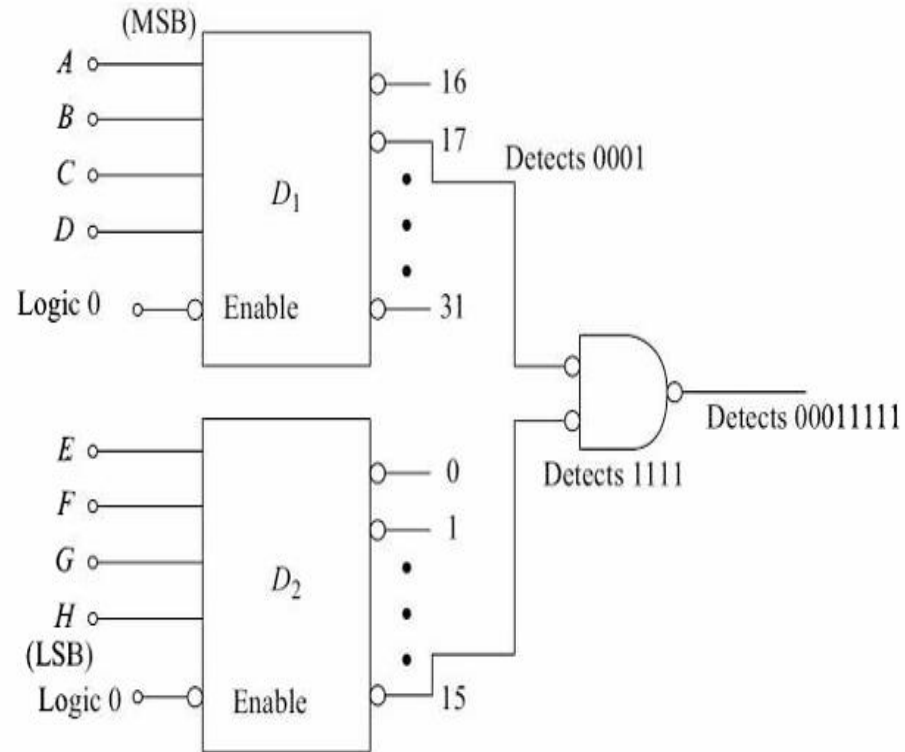


Fig. 6.11 An Alternative Approach to Decode Some Combinations

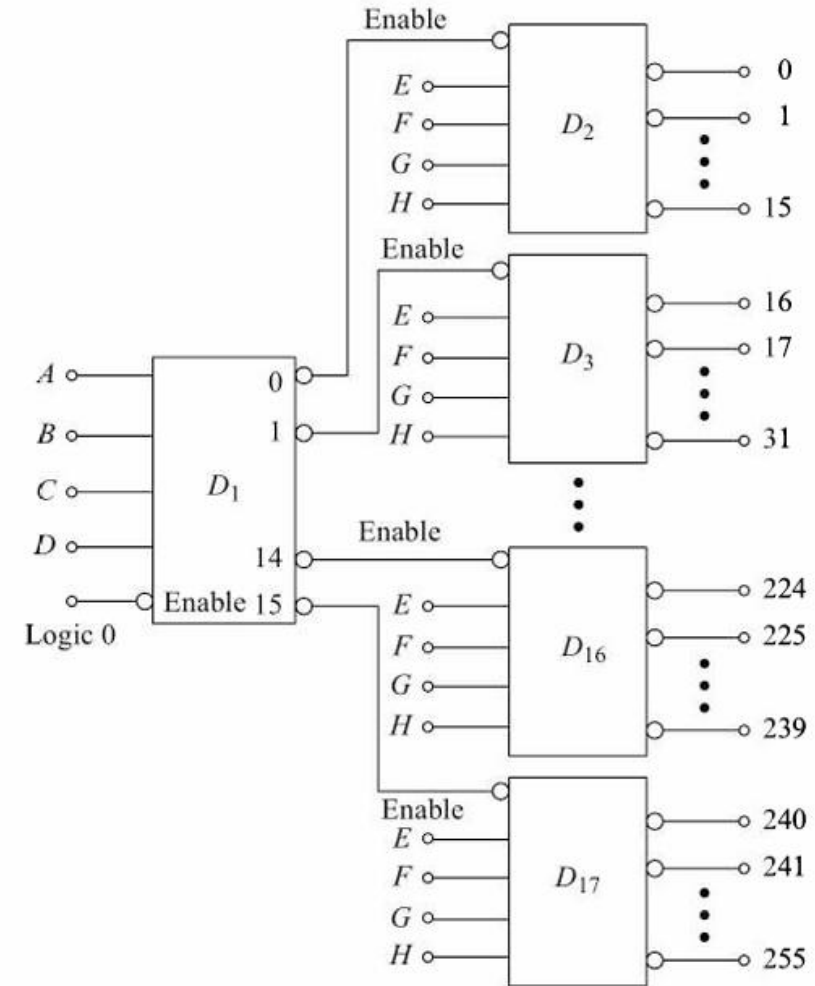


Fig. 6.10 8-Line-to-256-Line Decoder Using 4-Line-to-16-Line Decoders Tree

USE OF MSI DEVICES FOR MULTIPLEXER

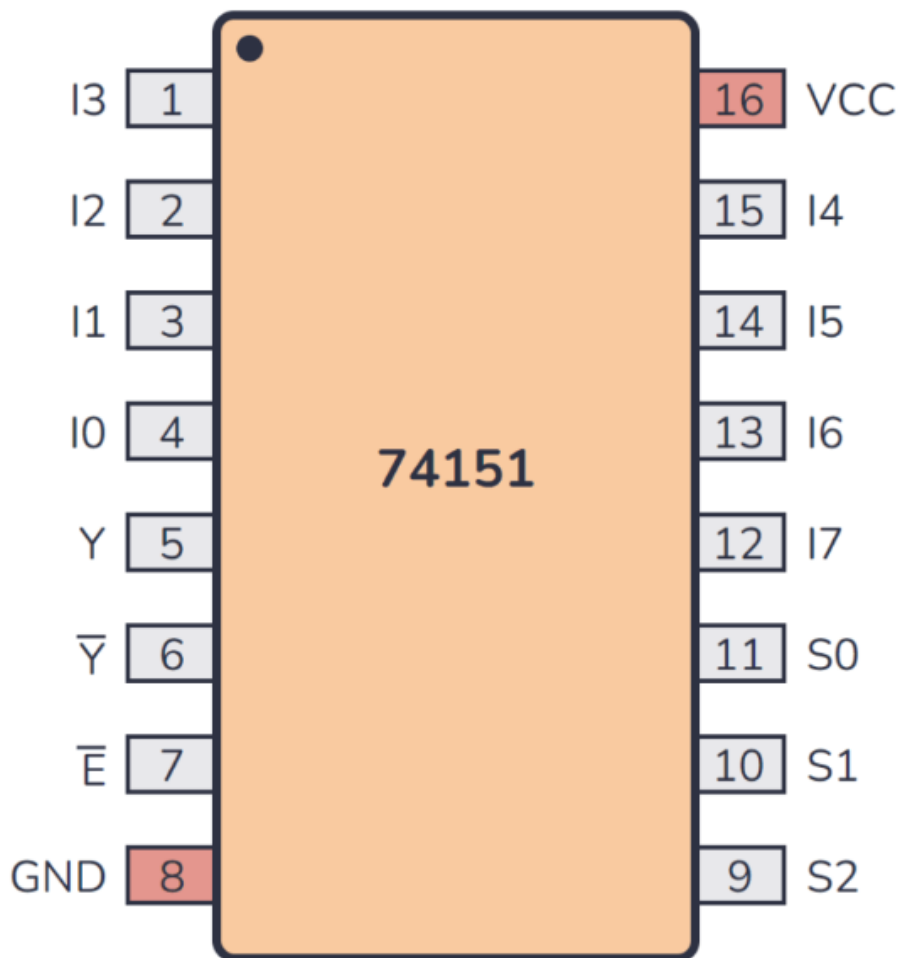
Table 6.2 *Available Multiplexer ICs*

IC No.	Description	Output
74157	Quad 2:1 Multiplexer	Same as input
74158	Quad 2:1 Multiplexer	Inverted input
74153	Dual 4:1 Multiplexer	Same as input
74352	Dual 4:1 Multiplexer	Inverted input
74151	8:1 Multiplexer	Complementary outputs
74152	8:1 Multiplexer	Inverted input
74150	16:1 Multiplexer	Inverted input

USE OF MSI DEVICES FOR MULTIPLEXER

- IC 74151 is an 8-input multiplexer integrated circuit (IC).
- It is a CMOS chip that can be used to select one of eight input signals and output it to a single output pin.
- The IC has three select inputs, which are used to select the input signal that will be output.
- The select inputs can be wired to a decoder to create a binary code that selects the input signal.
- The IC also has two complementary outputs, which provide both inverting and non-inverting buffer operation.
- This means that the output signal can be inverted or not, depending on the configuration of the IC.
- The IC74151 is a versatile device that can be used in a variety of applications. It is commonly used in digital logic circuits, such as Boolean function generators and decoders. It can also be used in analog circuits, such as sample-and-hold circuits and voltage comparators

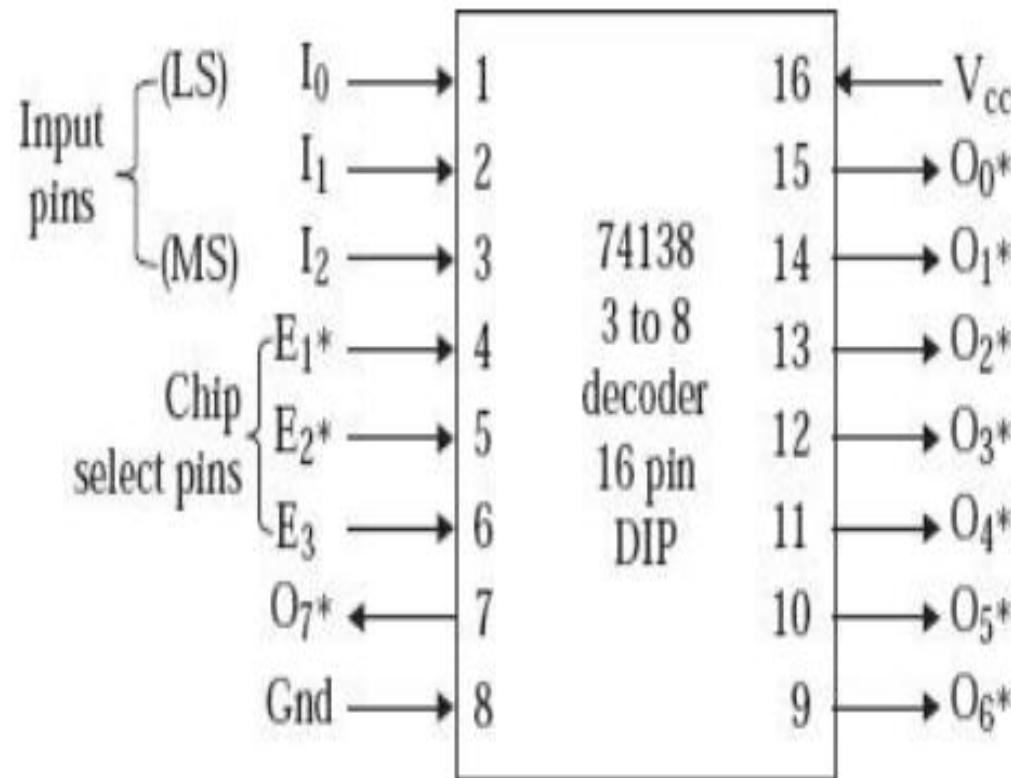
USE OF MSI DEVICES FOR MULTIPLEXER



Pin Name	Pin #	Type	Description
I3	1	Input	Data input 3 for the 8-to-1 multiplexer.
I2	2	Input	Data input 2 for the 8-to-1 multiplexer.
I1	3	Input	Data input 1 for the 8-to-1 multiplexer.
I0	4	Input	Data input 0 for the 8-to-1 multiplexer.
Y	5	Output	Output from the 8-to-1 multiplexer.
$\bar{Y}$	6	Output	Complementary (inverted) output from the multiplexer.
$\bar{E}$	7	Input	Enable input (active low).
GND	8	Power	Connect to ground (GND).
S2	9	Input	Select input S2 for the multiplexer.
S1	10	Input	Select input S1 for the multiplexer.
S0	11	Input	Select input S0 for the multiplexer.
I7	12	Input	Data input 7 for the 8-to-1 multiplexer.
I6	13	Input	Data input 6 for the 8-to-1 multiplexer.
I5	14	Input	Data input 5 for the 8-to-1 multiplexer.
I4	15	Input	Data input 4 for the 8-to-1 multiplexer.
VCC	16	Power	Positive power supply. Connect to +5V power.

USE OF MSI DEVICES FOR DECODER

- is a 3 to 8 line decoder integrated circuit from the 74xx family of transistor-transistor-logic-gates.
- The main function of this IC is to decode otherwise demultiplex the applications.
- The setup of this IC is accessible with 3-inputs to 8-output setup. This IC is mainly used in applications like memory decoding with high performance otherwise data routing, etc.
- These ICs can be used for minimizing the system decoding effects in memory systems with high performance.
- This IC includes three enable pins (where two pins are active low and one is active high) decreases the necessity of outside gates



Magnitude Comparator

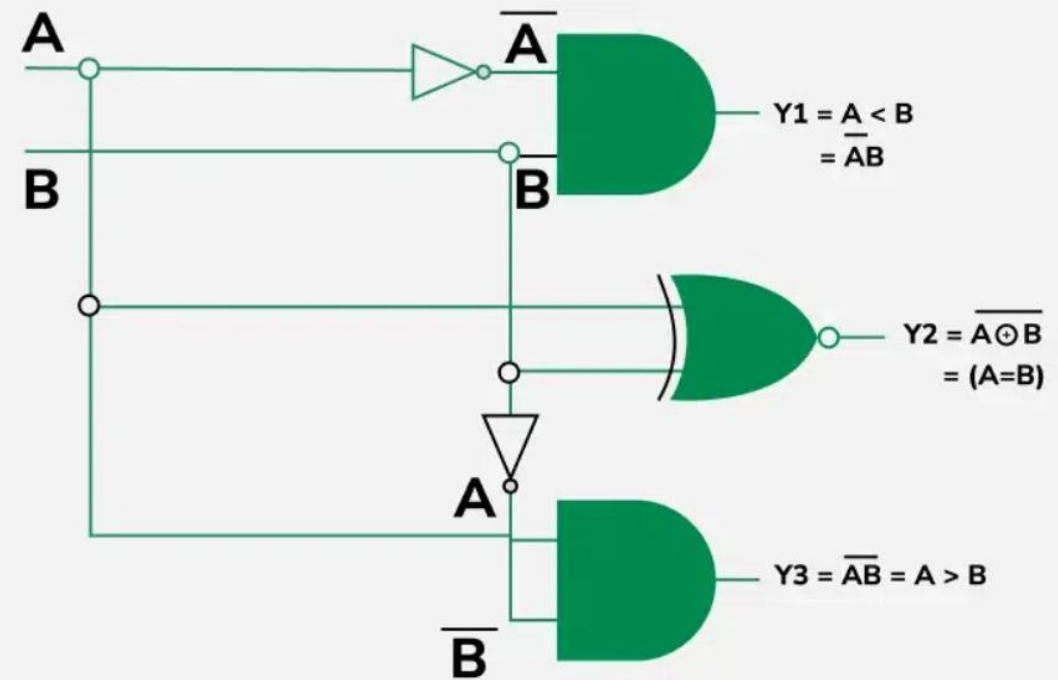
- It Basically compares two binary numbers and determines their relative magnitude.
- It gives output whether one number is greater than the other, or less than or equal.
- These comparators are used in digital systems, such as for sorting networks, and decision-making circuits to handle numerical comparisons perfectly without any error.
- The circuit works by comparing the bits of the two numbers starting from the most significant bit (MSB) and moving toward the least significant bit (LSB).
- At each bit position, the two corresponding bits of the numbers are compared.
- If the bit in the first number is greater than the corresponding bit in the second number, the $A > B$ output is set to 1, and the circuit immediately determines that the first number is greater than the second. Similarly, if the bit in the second number is greater than the corresponding bit in the first number, the $A < B$ output is set to 1, and the circuit immediately determines that the first number is less than the second. If the two corresponding bits are equal, the circuit moves to the next bit position and compares the next pair of bits. This process continues until all the bits have been compared.
- If at any point in the comparison, the circuit determines that the first number is greater or less than the second number, the comparison is terminated, and the appropriate output is generated.
- If all the bits are equal, the circuit generates an $A = B$ output, indicating that the two numbers are equal.

1-Bit Magnitude Comparator

A	B	A < B	A = B	A > B
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

1-Bit Magnitude Comparator

1 Bit Magnitude Comparator



2-Bit Magnitude Comparator

INPUT				OUTPUT		
A1	A0	B1	B0	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

Figure-4: Truth Table of 2-Bit Comparator

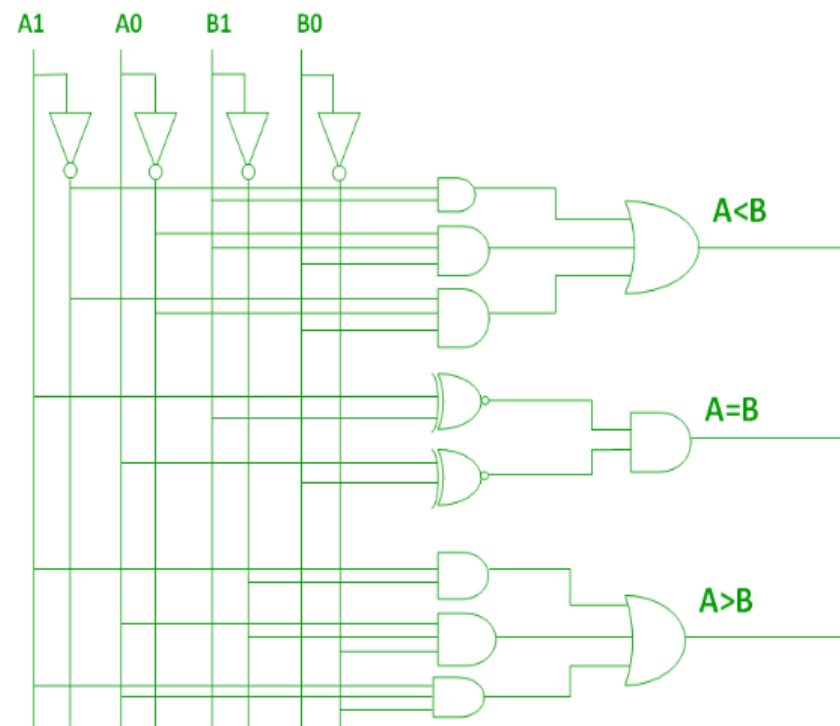


Figure-5: Logic Circuit of 2-Bit Comparator

2-Bit Magnitude Comparator

Truth Table of Output $A < B$

		A < B			
		B1B0	00	01	11
A1A0	00	0	1	1	1
	01	0	0	1	1
	11	0	0	0	0
	10	0	0	1	0

2-Bit Magnitude Comparator

		A > B			
		B1B0			
A1A0		00	01	11	10
	00	0	0	0	0
	01	1	0	0	0
	11	1	1	0	1
	10	1	1	0	0

Truth Table of Output A > B

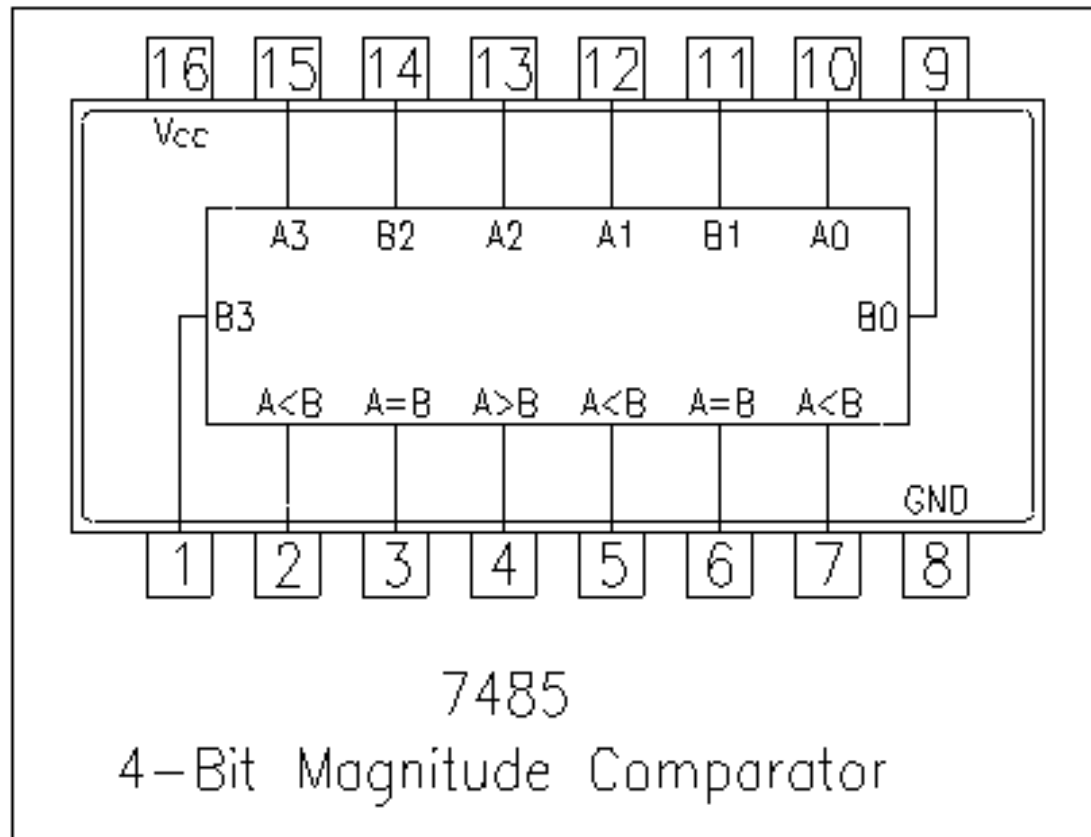
		A = B			
		B1B0			
A1A0		00	01	11	10
	00	1	0	0	0
	01	0	1	0	0
	11	0	0	1	0
	10	0	0	0	1

Truth Table of Output A = B

4-Bit Magnitude Comparator

- A comparator used to compare two binary numbers each of four bits is called a 4-bit magnitude comparator.
- It consists of eight inputs each for two four-bit numbers and three outputs to generate less than, equal to, and greater than between two binary numbers.
- The IC 7485 is used as a 4-bit magnitude comparator.
- The total input bits present are 8 bits so the truth table has $2^8 = 256$ entries in the truth table instead a functional table is used to understand the working of the magnitude comparator.

4-Bit Magnitude Comparator



Pin Number	Description
1	B3 Input
2	A<B Input
3	A=B Input
4	A>B Input
5	A>B Output
6	A=B Output
7	A<B Output
8	Ground
9	B0 Input
10	A0 Input
11	B1 Input
12	A1 Input
13	A2 Input
14	B2 Input
15	A3 Input
16	Vcc - Positive Supply

4-Bit Magnitude Comparator

Table 6.9 *Function Table of 7485 4-Bit Comparator*

Comparing inputs A, B	Cascading inputs			Outputs		
	$A > B$	$A = B$	$A < B$	$A > B$	$A = B$	$A < B$
$A > B$	X	X	X	1	0	0
	1	0	0	1	0	0
	X	1	X	0	1	0
$A = B$	0	0	1	0	0	1
	0	0	0	1	0	1
	1	0	1	0	0	0
$A < B$	X	X	X	0	0	1

4-Bit Magnitude Comparator

- The table 6.9 represents the function table of a magnitude comparator with cascading inputs, typically used in digital electronics (for example in ICs like the 4-bit magnitude comparator).
- It shows how the comparator determines whether $A < B$, $A = B$, or $A > B$ while also considering results from previous stages in cascading.

Cascading Inputs

- ❖ These inputs come from the **previous comparator stage** when multiple comparators are connected together.
- ❖ **$I(A < B)$** → Previous stage determined $A < B$
- ❖ **$I(A = B)$** → Previous stage determined $A = B$
- ❖ **$I(A > B)$** → Previous stage determined $A > B$
- ❖ These allow comparators to be **connected in series to compare larger binary numbers**

4-Bit Magnitude Comparator

Case-by-Case Explanation

1. Case 1: $A < B$

- If the current bits show $A < B$, the comparator output becomes

$I(A < B)$	$I(A = B)$	$I(A > B)$	Result
X	X	X	$Y(A < B) = 1$

- Meaning: Regardless of previous stages, A is already smaller.
- So output remains $A < B$.

4-Bit Magnitude Comparator

Case-by-Case Explanation

2. Case 2: $A = B$

When previous stage says $A > B$

$I(A < B)$	$I(A = B)$	$I(A > B)$	Result
0	0	1	$A > B$

When previous stage says $A = B$

$I(A < B)$	$I(A = B)$	$I(A > B)$	Result
X	1	X	$A = B$

4-Bit Magnitude Comparator

Case-by-Case Explanation

2. Case 2: $A = B$

- If A equals B in this stage, the result depends on previous cascade inputs.
- This means: If previous bits were equal and current bits are equal the overall numbers are equal.

When previous stage says $A < B$

$I(A < B)$	$I(A = B)$	$I(A > B)$	Result
1	0	0	$A < B$



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



THANK YOU